

COMPUTER GRAPHICS (CAC305) LECTURE 1

Er. Sanim Kumar Khatri

CONTENT

- Course Roadmap
- Computer Graphics (Introduction)
- Applications of Computer Graphics
- Recent trends
- Basic Terminologies

Syllabus

Unit 1: Introduction

6

- 1.1 Advantage of Computer Graphics and Areas of Applications
- 1.2 Hardware and Software for Computer Graphics. (Hard Copy, Display Technologies),
- 1.3 Random Scan Display System, Video Controller, Random Scan Display Processor
- 1.4 Raster Graphics
- 1.5 Scan Conversion Algorithms (Line, Circle, Ellipse)
- 1.6 Area Filling (Rectangle, Ellipse), Clipping (Lines, Circle, Ellipse), Clipping Polygons

Unit 2: Two dimensional and three dimensional transformations

7

- 2.1 2-Dimensional transformation
- 2.2 2-D Translation, Rotation, Scaling,
- 2.3 Homogeneous Coordinates, Reflection, Shear transform
- 2.4 3-dimensional transformation,
- 2.5 3-D Translation, Rotation Scaling, Reflection, Shear.

3 Unit 3: Clipping

7

- 3.1 Window to view port transformation
- 3.2 Clipping, line clipping,
- 3.3 Cohen –Sutherland line clipping
- 3.4 Polygon clipping
- 3.5 Sutherland and Gary Hodgman polygon clipping algorithm

Unit 4: Visible Surface Determination and computer graphics algorithm

15

- 4.1 Image space and object space techniques
- 4.2 Hidden Surface removal—Depth comparison
- 4.3 Z-Buffer Algorithm
- 4.4 Back-Face Removal
- 4.5 The Painter's Algorithm
- 4.6 Scan-Line Algorithm
- 4.7 Light and Color and different color models (RGB, CMY, YIQ)

Unit 5: Animation and virtual reality

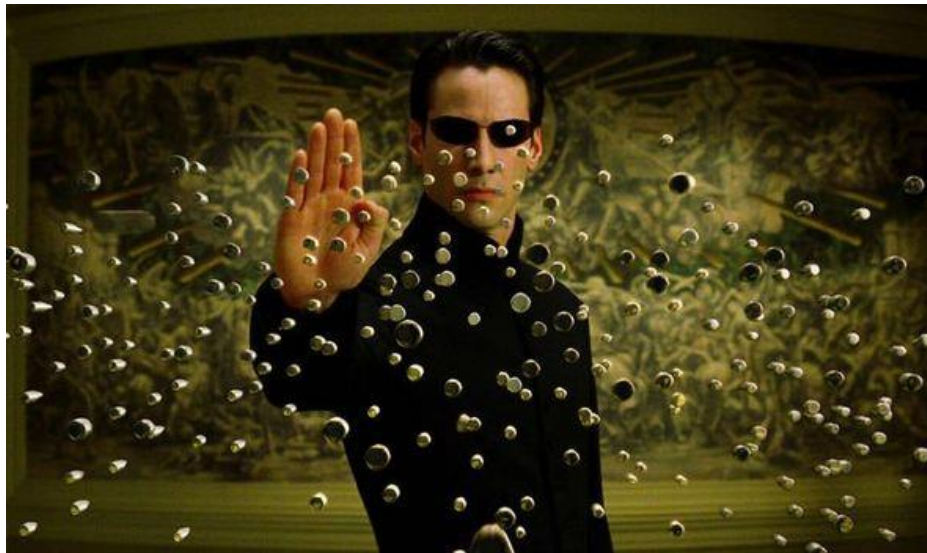
10

- 5.1 Basic Principles of Animation and Types of Animation
- 5.2 Introduction to the flash interface
- 5.3 Setting stage dimensions, working with panels, panel layouts
- 5.4 Layers & Views
- 5.5 Shaping Objects – Overview of shapes, Drawing & Modifying Shapes
- 5.6 Bitmap Images & Sounds
- 5.7 Animation -Principles , Frame by frame animation, tweening, masks
- 5.8 Introduction to virtual reality

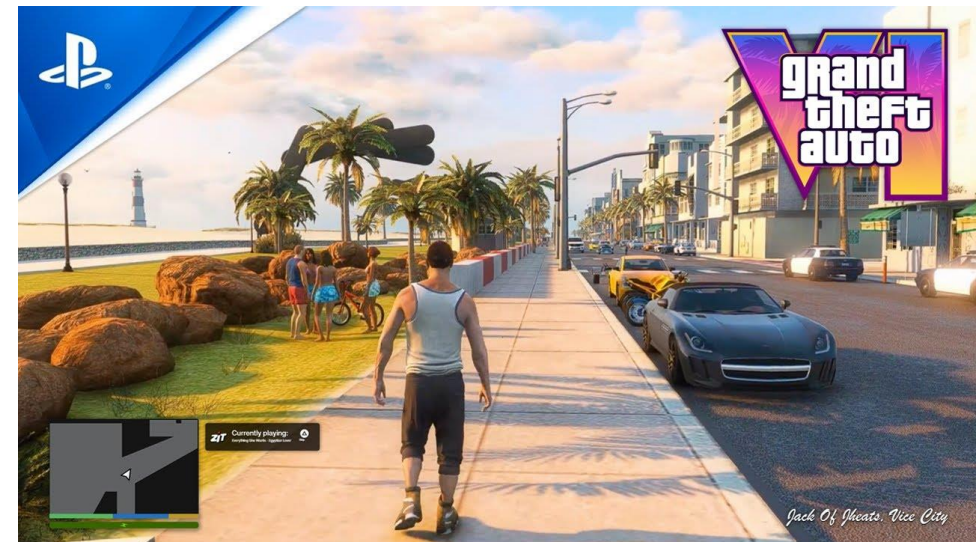
Reference Books

- Donald Hearn and M. Pauline Baker: Computer Graphics C Version, Prentice-Hall.
- James D. Foley, Andries van Dam, Steven K. Feiner, John F. Hughes, Computer Graphics: Principles and Practice in C, Addison-Wesley
- Mason Woo, Jackie Neider, Tom Davis, Dave Shreiner, "OpenGL Programming Guide", The Official Guide to Learning OpenGL, OpenGL Architecture Review Board, LPE Pearson Edition Asia
- Robert R & Snow D Flash CS4 Professional Bible, Wiley Publishing

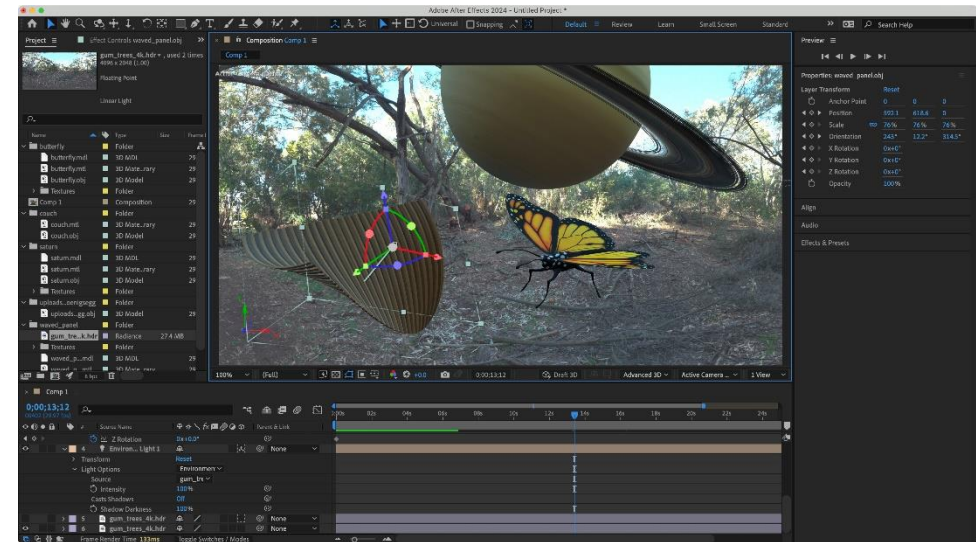
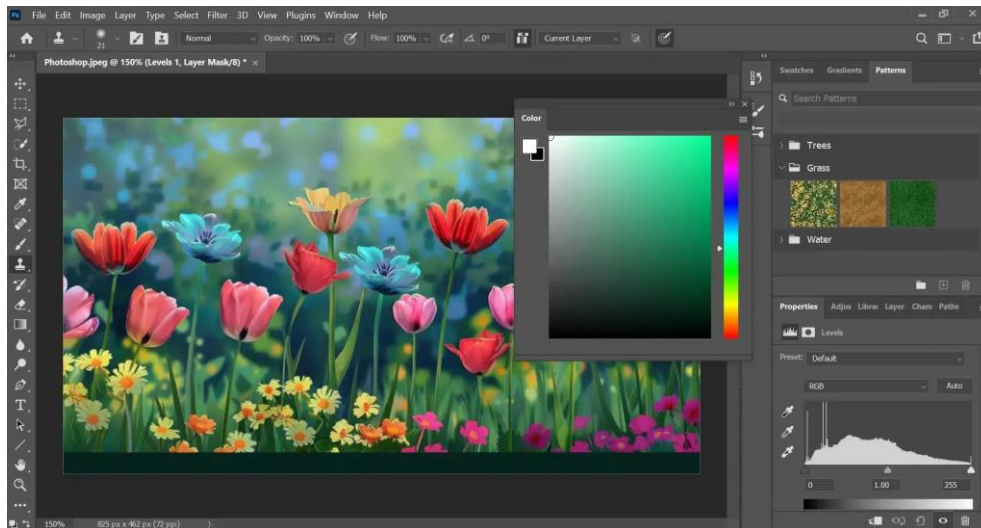
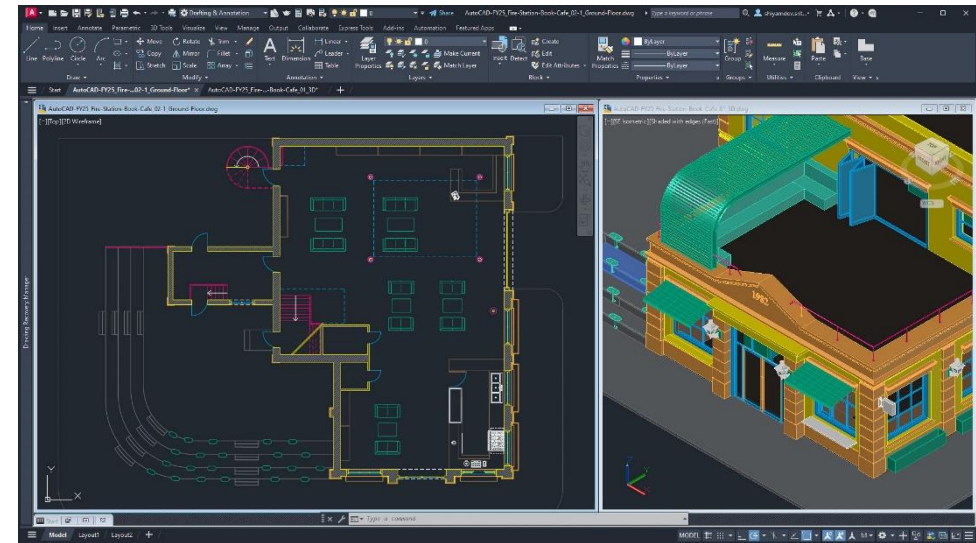
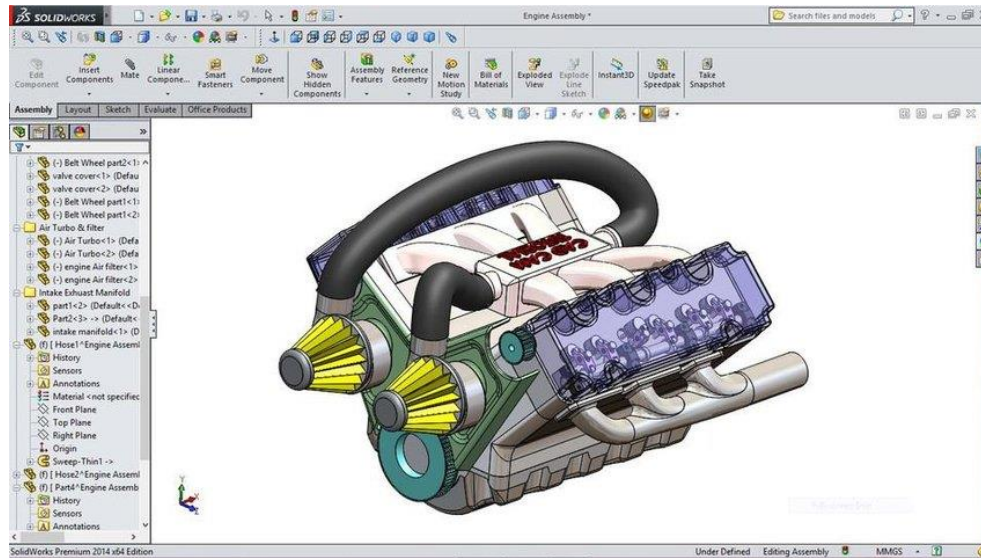
Introduction



Introduction



Introduction



Introduction

Computer Graphics

Computer + Graphics

Computer – computing tool

Graphics – visual representation

We find computer graphics used routinely in such diverse areas as science, engineering, medicine, business, industry, government, art, entertainment, advertising, education, and training.

Introduction

- Computer Graphics is a field related to the generation of graphics using computers.
- Computer graphics are graphics created using computers and the representation of image data by a computer specifically with help from specialized graphic hardware and software.
- It includes the creation, storage, and manipulation of images of objects.
- These objects come from diverse fields such as physical, mathematical, engineering, architectural, abstract structures and natural phenomenon.

Areas of applications

Following are the areas of applications of Computer Graphics

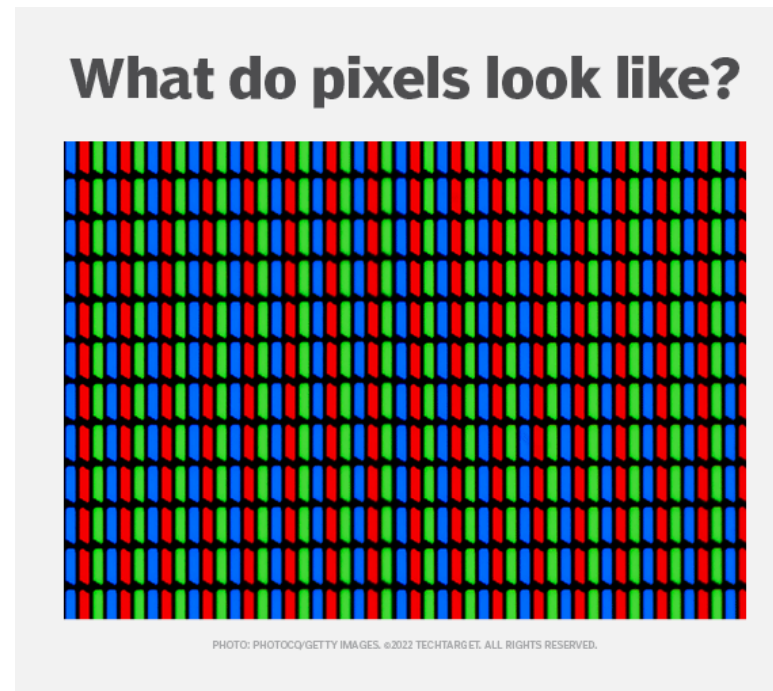
1. Computer Aided Design (CAD)
2. Presentation Graphics
3. Computer Art
4. Entertainment
5. Education and Training
6. Visualization
7. Image Processing
8. Graphical User Interfaces (GUI)
9. Simulation
10. Cartography

Recent Trends

1. AI in Graphics
 - AI-generated images/videos
2. VR/AR
 - Virtual Reality & Augmented Reality
 - Immersive gaming, simulation, training
3. Real-time Ray Tracking
 - High quality rendering in games and files.
4. Others
 - GPU advancement
 - Cloud Graphics
 - Procedural content

Pixels

- The full form of the pixel is “Picture Element”. It is also known as “PEL”.
- Pixel is the smallest element of an image on a computer display, whether they are LCD or CRT monitors.
- A screen is made up of a matrix of thousands or millions of pixels.
- A pixel is represented with a dot or a square on a computer screen.



Concept of Bits Per Pixel

- BPP or Bits per Pixel is used to denote the number of bits per pixel and the number depends on the depth of color.
- The number of distinct colors that can be represented by a pixel depends on the number of bits per pixel (bpp).
- A 1 bpp image uses 1 bit for each pixel, so each pixel can be either on or off. Each additional bit doubles the number of colors available, so a 2 bpp image can have 4 colors, and a 3 bpp image can have 8 colors.
- 16 bit -> high color
- 24 bit -> true color

2^1		→ 1 bit	→ 2 colors
2^2		→ 2 bit	→ 4 colors
2^3		→ 3 bit	→ 8 colors
2^4		→ 4 bit	→ 16 colors
2^5		→ 5 bit	→ 32 colors
2^6		→ 6 bit	→ 64 colors
2^7		→ 7 bit	→ 128 colors
2^8		→ 8 bit	→ 256 colors
2^{16}		→ 16 bit	→ 32,768 colors
2^{24}		→ 24 bit	→ 16,777,216 colors

Image storage requirements

Size of an Image = rows * cols * bpp

Example: Let row be 3000 and columns be 1687, and it has 256 shades of bpp.

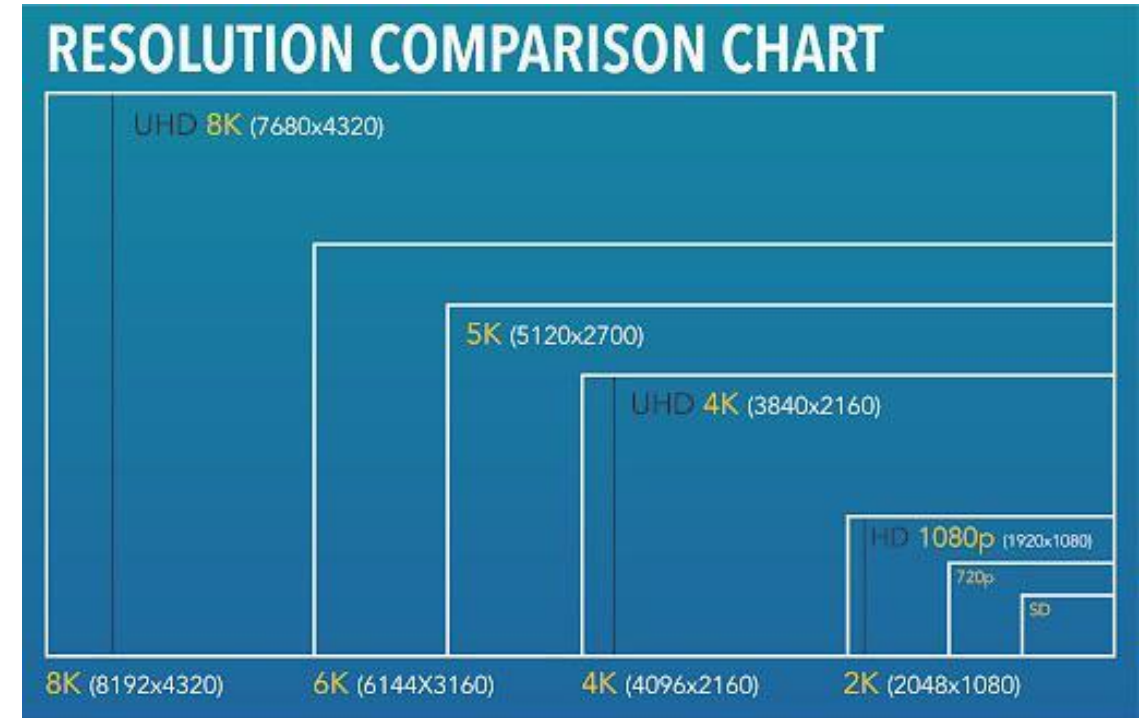
Size of an image = rows * cols * bpp
= 3000 * 1687 * 8
= 40488000 bits

As it is not a standard answer so we can convert it in the following ways:

Converting it into bytes = 40488000 / 8 = 5061000 bytes
 = 5061000 / 1024 = 4942 KB
 = 4942 / 1024 = 4 MB

Resolution

- The maximum number of points (pixel) that can be displayed without overlap on a CRT is referred to as the resolution.
- It is also defined as the number of points per unit of measure (per centimeter or per inch) that can be plotted horizontally and vertically.
- Resolution is defined as the maximum number of points that can be displayed horizontally and vertically without overlap on a display device.

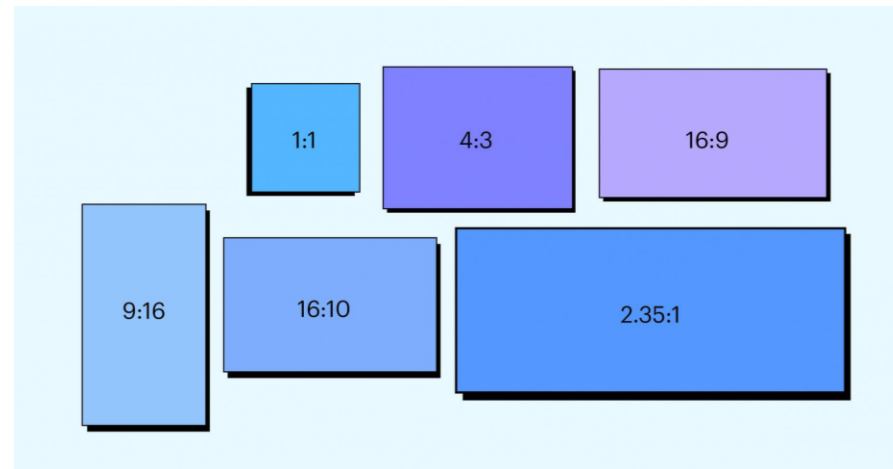


Aspect Ratio

- Aspect ratio is the ratio between height and width of an image.

$$\text{Aspect ratio} = \text{width (image unit)} / \text{height (image unit)}$$

- A colon is used to separate two numbers.
- It is commonly expressed as two numbers separated by a colon, as in 16:9. For an x:y aspect ratio, no matter how big or small the image is, if the width is divided into x units of equal length and the height is measured using this same length unit, the height will be measured to be y units.
- In, for example, a group of images that all have an aspect ratio of 16:9, one image might be 16 inches wide and 9 inches high, another 16 centimeters wide and 9 centimeters high, and a third might be 8 yards wide and 4.5 yards high.

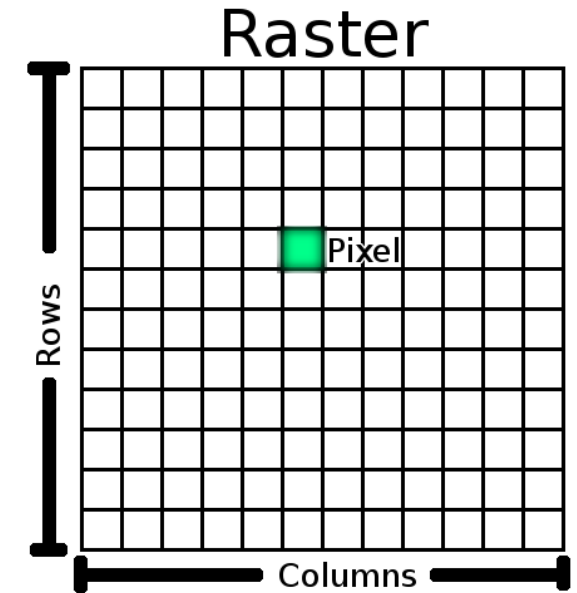


Raster

- A rectangular array of points or dots.
- A row of pixels is called Scan Line.

Persistence

- It means how long they continue to emit light after the electron beam is removed.
- Persistence is defined as the time it takes the emitted light from the screen to decay to one-tenth of its original intensity.
- Lower persistence phosphors require higher refresh rates to maintain a picture on the screen.
- A phosphor with lower persistence is useful for animation and a higher-persistence phosphor is useful for displaying highly complex static picture.
- Graphics monitors are usually constructed with the persistence 10 to 60 microseconds.



Refresh Rate

- The number of times the screen is redrawn each second.
- Higher refresh rates mean less flicker on the screen, which translates into less eyestrain.

Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 2

Er. Sanim Kumar Khatri

CONTENT

1.2 Hardware and Software for Computer Graphics

- Display Technologies
 - CRT
 - Modern Flat Panel Displays (LCD, LED, Plasma)
- Hard Copy devices
 - Plotters
 - Printers
 - 3D printers

Display Technologies

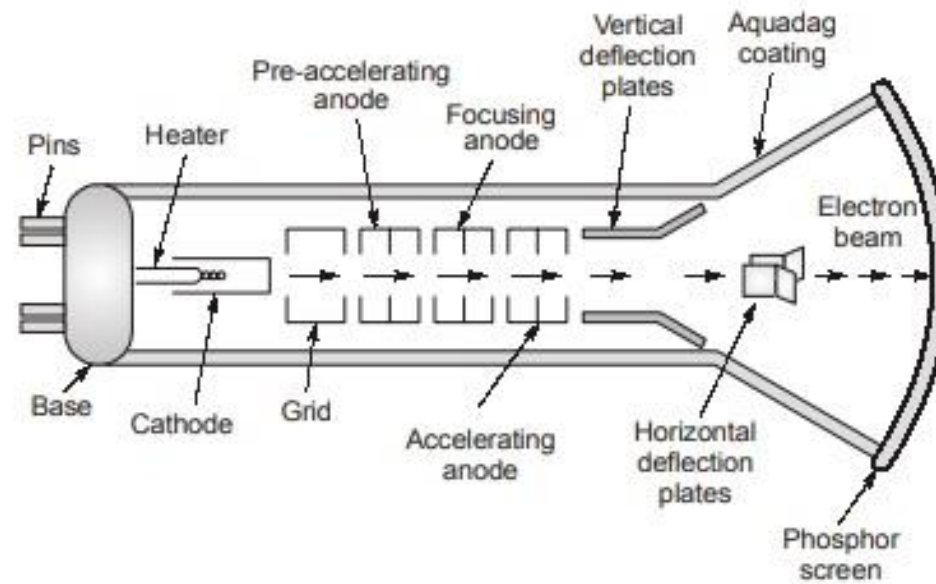


Fig. : Construction of a CRT

Cathode Ray Tube (CRT)

Working Principle

The original display technology. Its operation involves several key parts:

- An **Electron Gun** (cathode, grid) emits a high-velocity beam of electrons.
- A **Focusing System** narrows the beam into a sharp point.
- **Deflection System** (magnetic coils or electric plates) guides the beam to a specific point.
- The beam strikes a **Phosphor-Coated Screen**, causing the phosphor to glow and create a visible dot (pixel).

CRT: Scan Methods

Raster Scan

Sweeps the electron beam across the entire screen, one horizontal line at a time, from top to bottom.

- Picture is stored in a **Frame Buffer** as a grid of pixels.
- This is the standard for TVs and most monitors.
- Can display complex, realistic shaded images.

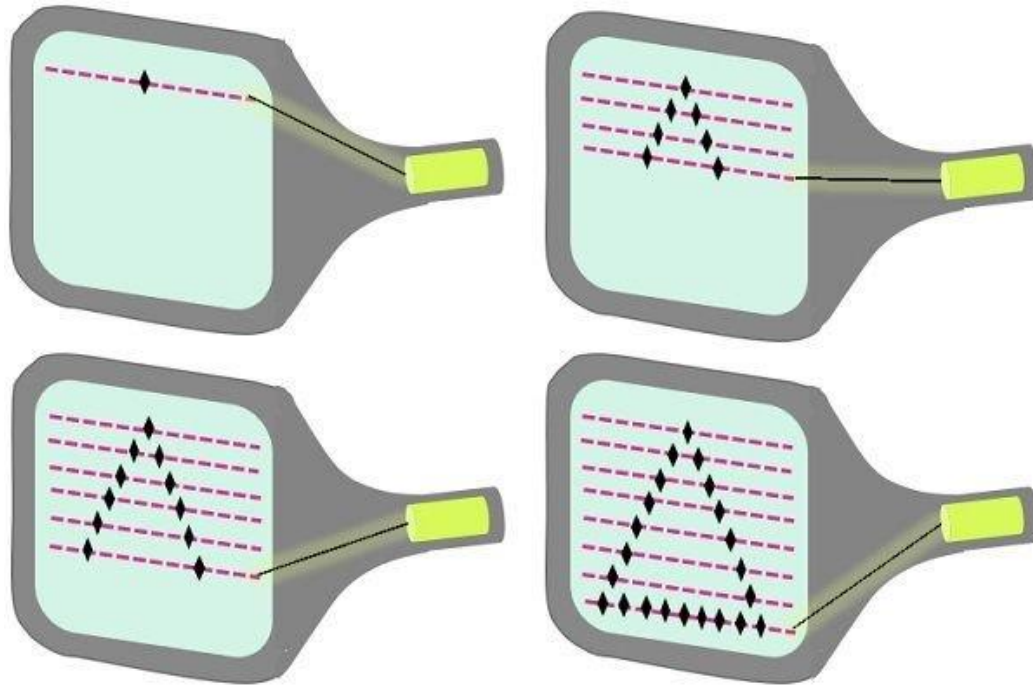
Random (Vector) Scan

Directs the electron beam only to the parts of the screen where the picture is to be drawn, like a pen.

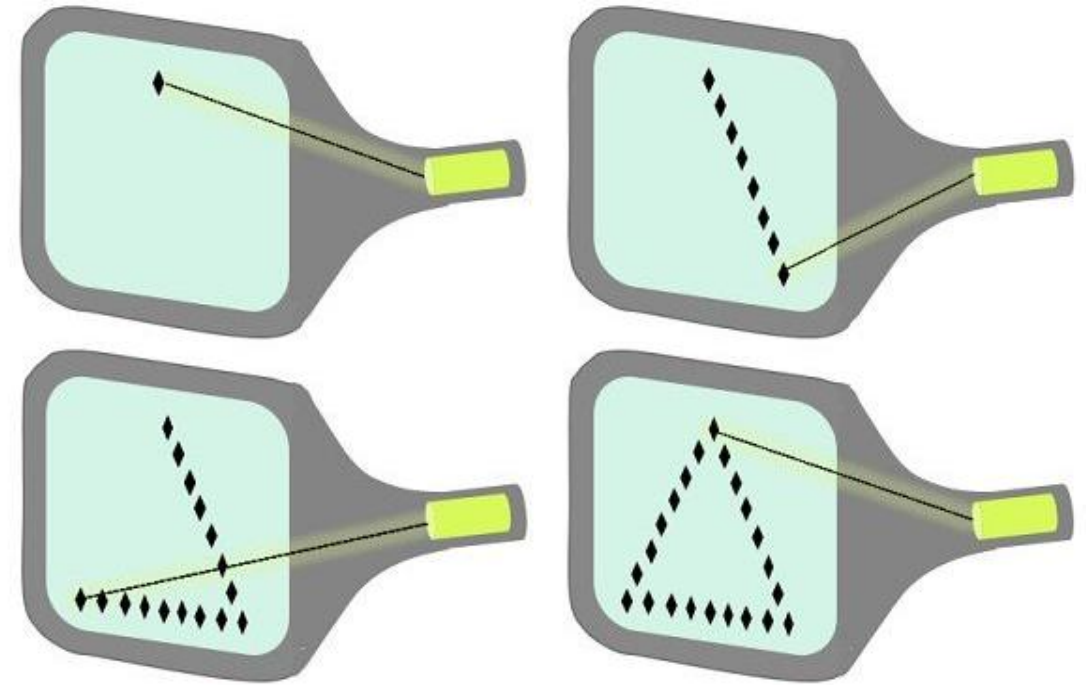
- Picture is stored as a set of **line-drawing commands** (a Display List).
- Produces smooth lines; used in early CAD systems.
- Cannot easily display filled areas or photos.

CRT: Scan Methods

Raster Scan



Random (Vector) Scan



Difference Between

Base of Difference	Raster Scan Display	Random (Vector) Scan Display
1. Electron Beam	The electron beam is swept across the screen one row at a time from top to bottom.	The electron beam is swept only to the parts of the screen where a picture is to be drawn.
2. Resolution	It has lower / poor resolution because picture definition is stored as an intensity value.	It has high resolution because picture definition is stored as a set of line commands.
3. Picture Definition	Picture definition is stored as a set of intensity values for all screen points (pixels) in a refresh buffer.	Picture definition is stored as a set of line instructions in a display list or file.
4. Realistic Display	Suitable for realistic display with shadow and color patterns due to ability to store pixel intensity values.	Designed mainly for line-drawing; cannot display realistic shaded scenes.
5. Image Drawing	Screen points or pixels are used to draw an image.	Mathematical functions are used to draw an image (line by line).
6. Cost	Cheaper than random scan display.	More expensive than raster scan display.

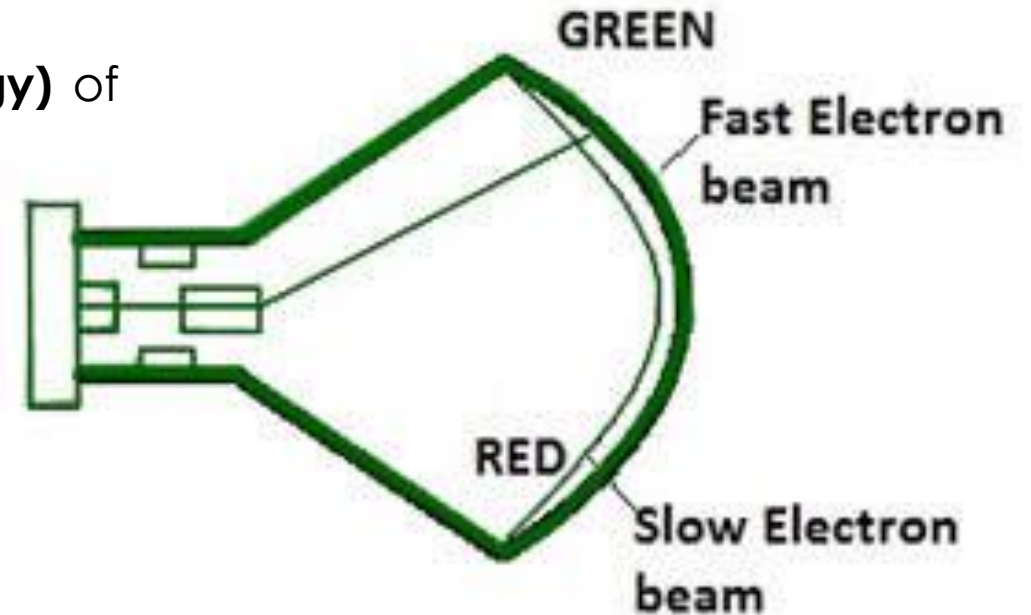
Difference Between

Base of Difference	Raster Scan Display	Random (Vector) Scan Display
7. Refresh Rate	60–80 fps.	Components drawn 30–60 times per second.
8. Interlacing	Uses interlacing.	Does not use interlacing.
9. Editing	Editing is difficult.	Editing is easy.
10. Refresh Area	Independent of picture complexity	Depends on picture complexity.
11. Smoothness	Produces jagged lines.	Produces smooth lines.

Color CRT: Beam Penetration

An early, simple method for color. It uses two different phosphor layers (e.g., red and green) coated on the screen.

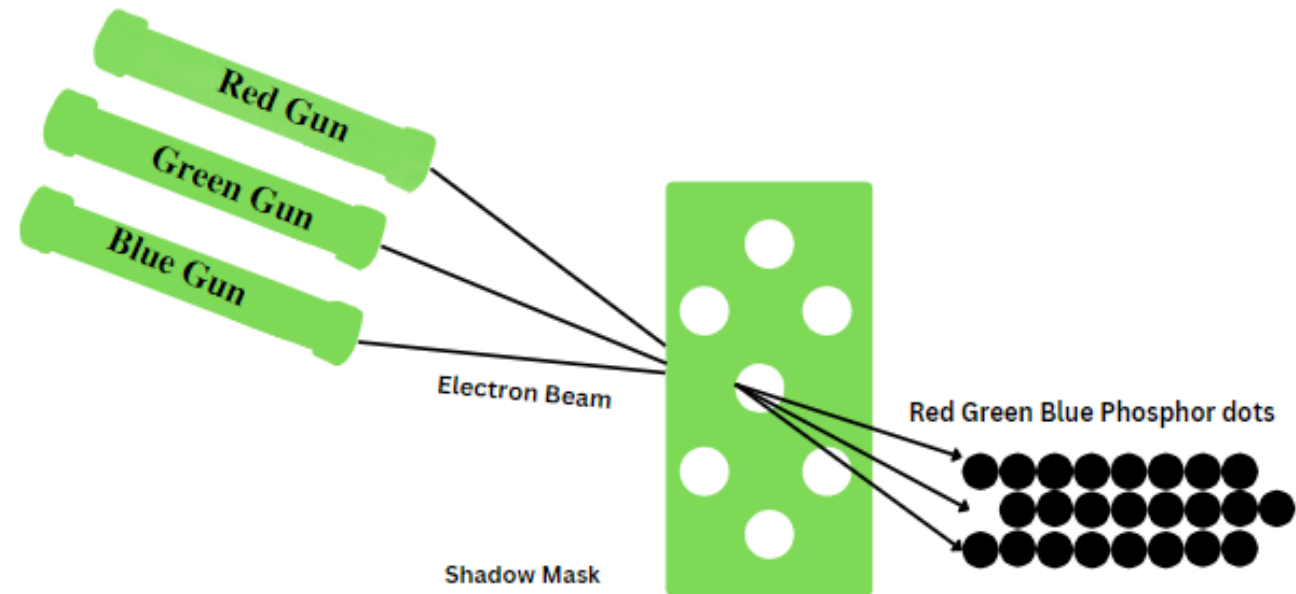
- The color displayed depends on the **speed (energy)** of the electron beam.
- **Slow Beam:** Excites only the outer (Red) layer.
- **Fast Beam:** Penetrates to the inner (Green) layer.
- **Intermediate Speed:** Mixes colors (Orange/Yellow).
- **Limitation:** Very limited color palette.



Color CRT: Shadow Mask

The most common method for producing color in CRTs. It uses three separate electron guns (Red, Green, Blue).

- The screen is coated with **phosphor dot triads** (one R, G, and B dot in a group).
- A **Shadow Mask**, a metal plate with tiny holes, is placed just behind the screen.
- The mask ensures each electron beam can only hit its corresponding color dot.
- By varying the intensity of the three beams, any color can be created.
- 2 types of arrangements are possible:
 - In-line Method
 - Delta delta method (Triad arrangement)



Delta- Delta method

- Delta- Delta method is commonly used in color CRT system
- It is called *Delta-Delta* because the three phosphor dots (R, G, B) are arranged in a triangular shape, similar to the Greek letter Δ (delta)."
- A shadow mask is often used to ensure that the electron beams from the guns fall on the correct phosphors
- The three electron beams are deflected and focused as a group onto the shadow mask
- When the three beams pass through a hole in the shadow mask, they activate a dot triangle, which appears as a small color spot on the screen.
- The phosphor dots in the triangles are arranged so that each electron beam can activate only its corresponding color dot when it passes through the shadow mask.
- Color variations in a shadow-mask CRT are obtained by varying the intensity levels of the three electron beams
- A white (or gray) area is the result of activating all three dots with equal intensity
- A black is produced with zero intensity of all colors
 - Yellow=Red+ Green
 - Magenta=Red+ Blue
 - Cyan= Blue +Green

Inline arrangement

- 3 electron guns and the corresponding red - green-blue color dots on the screen, are aligned along one scan line rather than in a triangular pattern.
- This inline arrangement of electron guns is easier to keep in alignment and is commonly used in high-resolution color CRT's
- In some low-cost systems, the electron beam can only be set to on or off, limiting displays to eight colors.
- More sophisticated systems can set intermediate intensity levels for the electron beams, allowing several million different colors to be generated
- Color CRTs in graphics systems are designed as RGB monitors. These monitors use shadow mask method and take the intensity level for each gun.
- A RGB color system with 24 bits of storage per pixel is known as full color system or true color system.

Color CRT

Feature	Delta-Delta	Inline
Phosphor pattern	Triangle (Δ)	Straight line (RGB)
Gun placement	Triangle	Horizontal line
Image sharpness	Less sharp	More sharp
Used in	Old color TVs	Computer monitors, modern CRT TVs

G
R B

R G B

Flat-Panel Displays (FPDs)

Modern displays that are thinner, lighter, and more power-efficient than CRTs. They are divided into two main categories:

- **Emissive Displays**

The pixels themselves convert electrical energy into light.

Examples: Plasma, LED, and OLED.

- **Non-Emissive Displays**

Use optical effects to modify external light (from a backlight or ambient light).

Example: LCD (Liquid Crystal Display).

- Backlight is the light source placed behind an LCD screen that provides illumination, since LCDs cannot emit light on their own.
- Ambient light means the natural or artificial light already present in the environment around a display. It is the *background light* of the room.

FPDs: How They Work

LCD (Liquid Crystal)

Uses a backlight. Liquid crystals "twist" or "untwist" to block or pass light through red, green, and blue color filters.

Plasma

Uses small cells of electrified gas (plasma) that discharge to make phosphors glow, producing a color. (Largely obsolete).

LED / OLED

LED: A type of LCD that uses efficient LEDs as its backlight.

OLED: Each pixel is an Organic LED that emits its own light (no backlight needed).

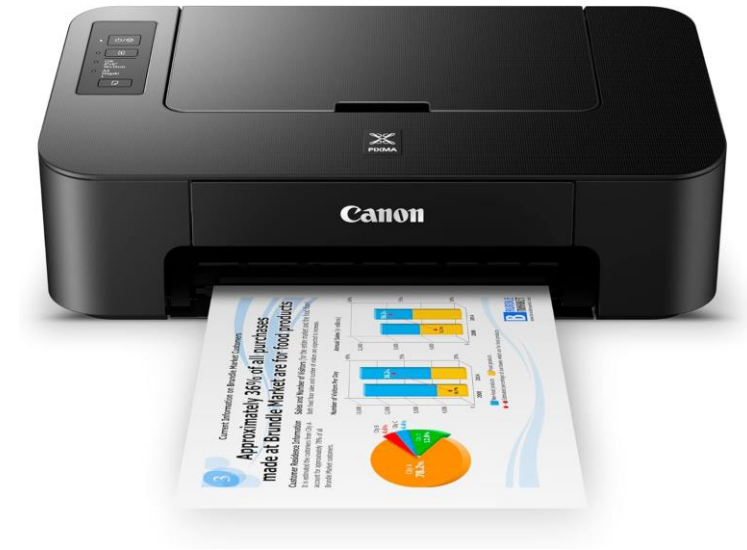
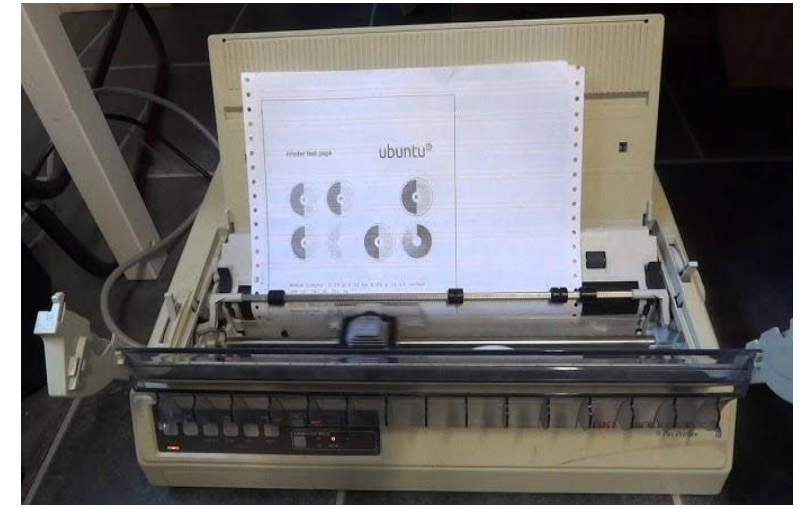
Display Technology Comparison				
Feature	CRT	LCD	LED (OLED)	Plasma
Principle	Electron Beam	Light Blocking	Light Emitting	Gas Discharge
Size/Weight	Bulky, Heavy	Thin, Light	Thinnest, Lightest	Thin, Heavy-ish
Contrast	Good	Good	Best (True Black)	Excellent
Power Use	High	Low	Lowest	High
Issues	Burn-in, Flicker	Viewing Angle	Lifespan (Blue)	Burn-in, Heat

Hard Copy Devices

Printers

Devices that produce text and graphics on a physical medium like paper.

- **Impact Printers (e.g., Dot Matrix)**
A physical head strikes an ink ribbon against the paper.
Noisy, but good for multi-part carbon-copy forms.
- **Non-Impact Printers (e.g., Inkjet, Laser)**
Inkjet: Sprays microscopic droplets of ink.
Laser: Uses a laser to create a static charge on a drum, which attracts toner and transfers it to paper.



Plotters

A specialist device that produces **vector graphics** drawings, rather than pixel-based images.

- Historically used in CAD, architecture, and engineering for large, precise line drawings.
- **Pen Plotters** use one or more automated pens to draw continuous lines on paper.
- Directly follows vector commands, similar to a Random Scan display.



3D Printers

Also known as **Additive Manufacturing**, this technology builds a physical 3D object from a digital model.

- **Working Principle:** Lays down successive layers of material (e.g., plastic, resin, metal) and fuses them together.
- Commonly used for rapid prototyping, creating custom parts, medical models, and hobbyist projects.



Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 3

Er Sanim Kumar Khatri

CONTENT

1.3 Raster and Random Scan Display System

- Raster Scan Display System
- Random Scan Display Processor
- Video Controller

Aliasing

The stair-step effect that appears on lines and boundaries due to the discrete nature of pixels in a raster display (the "jaggies").

Antialiasing

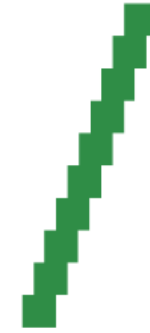
Techniques used to smooth the stair-step appearance of lines and edges by adjusting the intensity of pixels along the boundary.

Pixel Phasing (Sub-pixel Rendering)

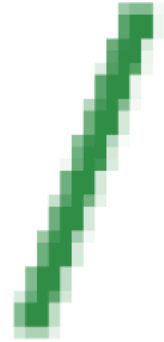
Pixel phasing refers to how the alignment of pixels in a display affects the appearance of an image, especially when sub-pixels (red, green, blue) are shifted slightly relative to each other.

Refresh Rate

The number of times per second the image on the screen is redrawn. Measured in Hertz (Hz). Higher rates reduce flicker.



Without Antialiasing



With Antialiasing

Interlacing

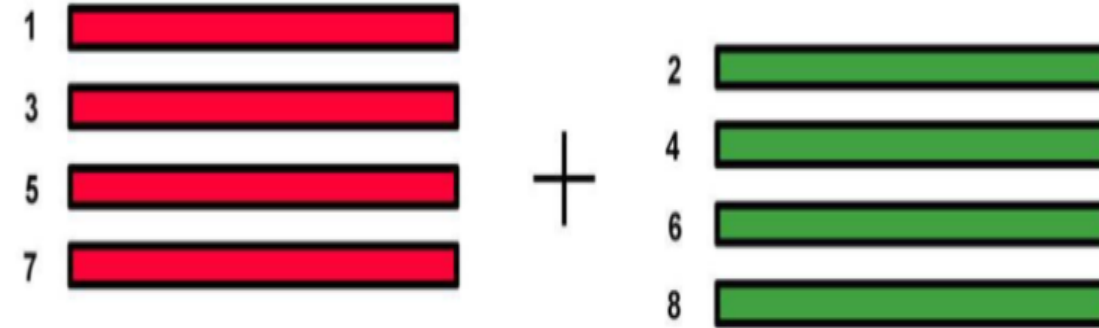
A display technique where alternating lines (odd field, then even field) are drawn in two passes to achieve higher perceived refresh rate with lower bandwidth.

Horizontal Retrace

The return of the electron beam (beam off) from the right side of a scan line to the start of the next line on the left.

Vertical Retrace

The return of the electron beam (beam off) from the bottom-right corner of the screen to the top-left corner to start a new frame.



Fluorescence or Phosphorescence

The property of the screen's phosphor coating to glow. Fluorescence is instantaneous; Phosphorescence is the continued light emission after the electron beam has passed (persistence).

Frame Buffer

A dedicated area of memory that holds the complete, digital representation of the image that will be displayed on the screen, organized by pixel coordinates.

Pixel Organization

Defines how color and intensity are stored. E.g., 1-bit (Monochrome), 8-bit (Indexed Color/256), 24-bit (True Color/RGB).

- b) What role does a frame buffer play in graphical rendering? A raster graphics system can produce a total number of 16777216 different level of intensities out of a single pixel on the screen, if the total resolution of the screen is 1280x800, what is the required size of the frame buffer of this system? 8

Q. A raster graphics system can produce a total number of 16777216 different level of intensities out of a single pixel on the screen, if the total resolution of the screen is 1280x800, what is the required size of the frame buffer of this system?

How much time is spent scanning across each row of pixels during screen refresh on a raster system with resolution 1024×768 and refresh rate 60 frames per second?

How much time is spent scanning across each row of pixels during screen refresh on a raster system with resolution 1024×768 and refresh rate 60 frames per second?

Resolution = 1024×768

Refresh rate = 60fps

For 60 frames, it takes 1 second to scan

For 1 frame, it takes $1/60$ second

1 frame = 1024×768 pixels

For 1024×768 pixels, it takes $1/60$ second

For scanning 1 pixel, it takes $1/(60 \times 1024 \times 768)$ second

For scanning 1 row of pixels = $1 \times 1024 / (60 \times 1024 \times 768)$ seconds
= 0.0000217013 seconds

Raster Scan Systems

Characteristics: Grid-Based

The screen is covered by a fixed grid of pixels (the raster).

Interactive raster graphics systems typically employ several processing units. In addition to the central processing unit, or CPU, a special-purpose processor, called the video controller or display controller, is used to control the operation of the display device.

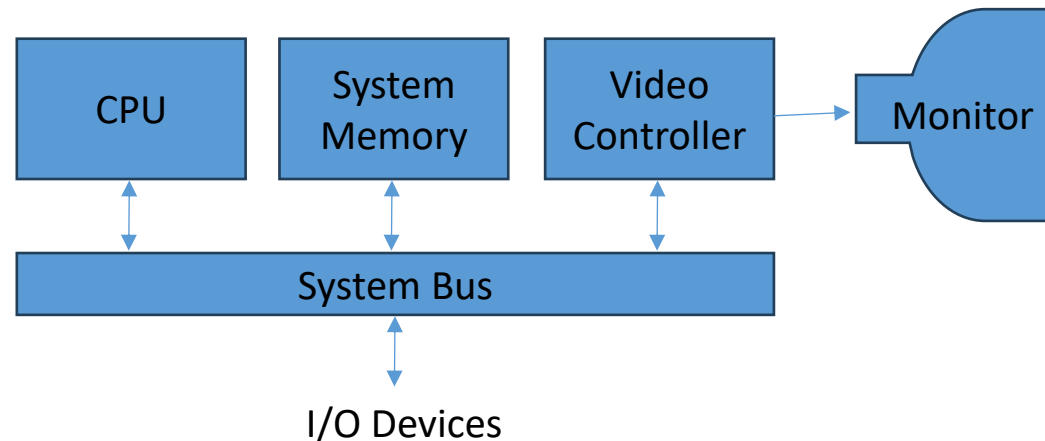


Figure: Architecture of a simple raster graphics system.

Raster Scan Systems

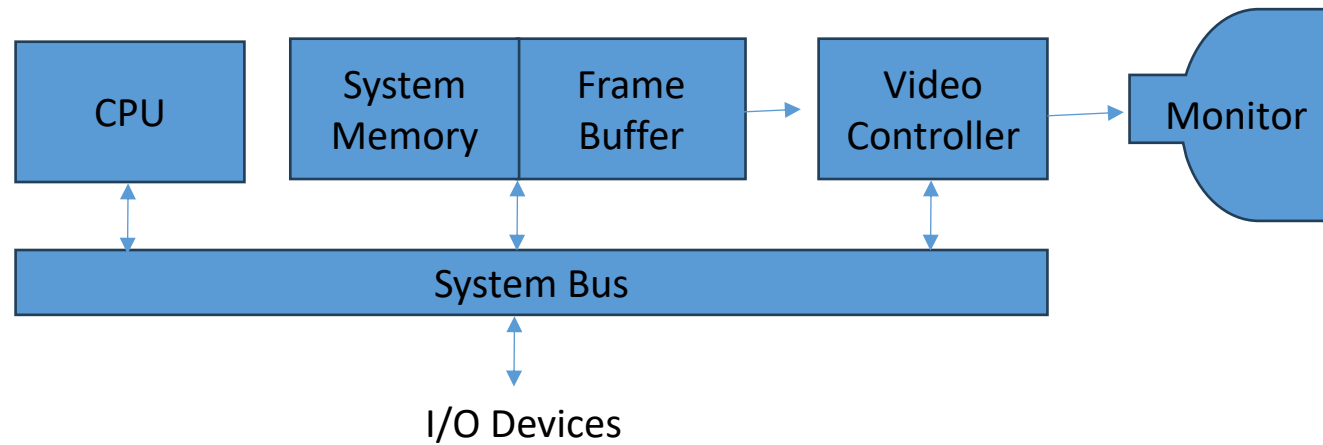


Figure: Architecture of a raster system with a fixed portion of the system memory reserved for the frame buffer.

Operation: The electron beam scans across every line, from top to bottom, regardless of the image content.

Memory: Requires a large **Frame Buffer** to store intensity and color data for every pixel.

Image Type: Ideal for realistic, complex images with solid colors, shading, and textures.

Raster Scan Systems

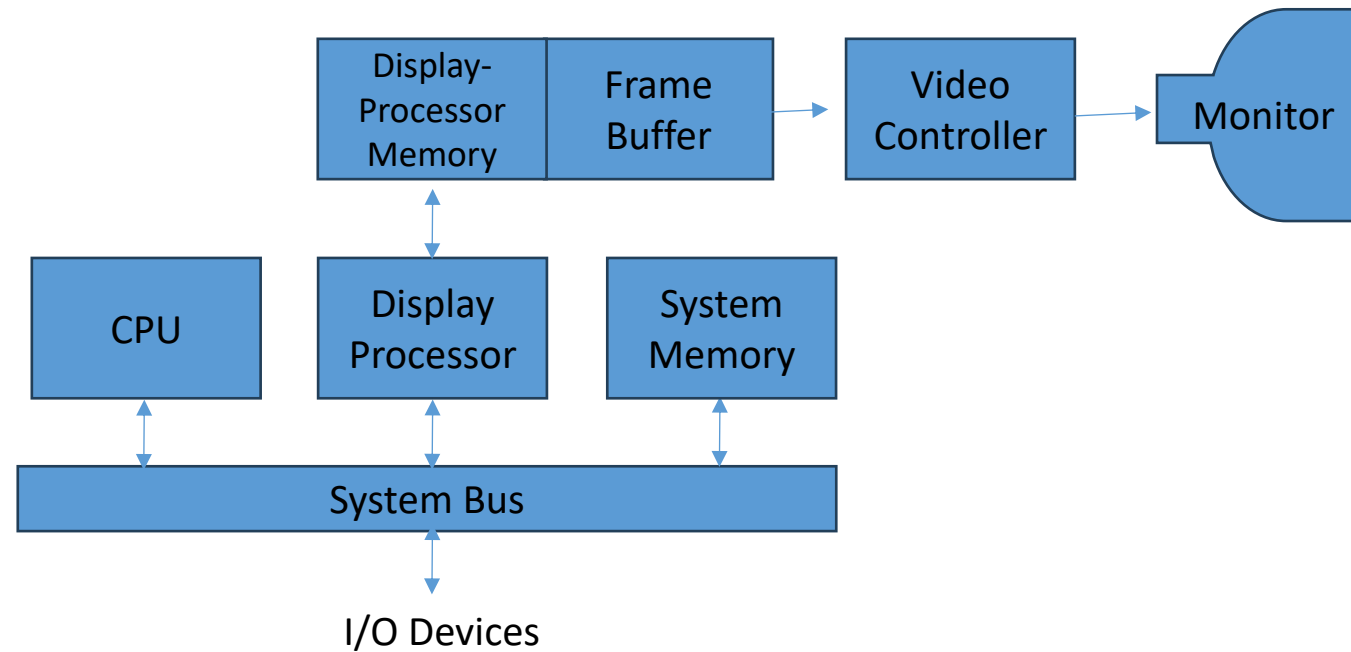


Figure: Architecture of a raster system with a display processor.

Random Scan Systems

Characteristics: Vector-Based

Also known as Vector, Stroke-Writing, or Calligraphic displays.

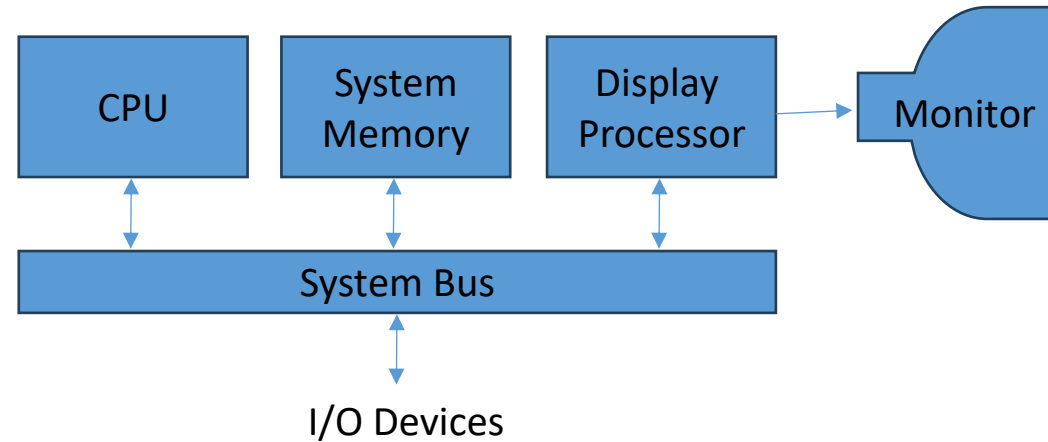


Figure: Architecture of a simple random graphics system.

Random Scan Systems

Operation: The electron beam is directed only to the areas where lines are to be drawn, like a plotter.

Memory: Stores a **Display File** (list of line-drawing commands), requiring minimal memory for simple scenes.

Image Type: Excellent for smooth, high-resolution line drawings, CAD/CAM, and simple wireframe models.

Difference

Feature	Raster Scan System	Random Scan System
Image Quality	Aliasing (jaggies) possible; full color/shading.	Very high resolution; perfectly smooth lines.
Display Data	Pixel intensities (Frame Buffer).	Line-drawing commands (Display File).
Refresh Rate	Constant (60Hz+), independent of scene complexity.	Variable, dependent on the number of lines drawn (flicker risk).
Cost/Complexity	More complex hardware (Frame Buffer, VRAM).	Simpler hardware, less memory required for vector graphics.

Video Controller

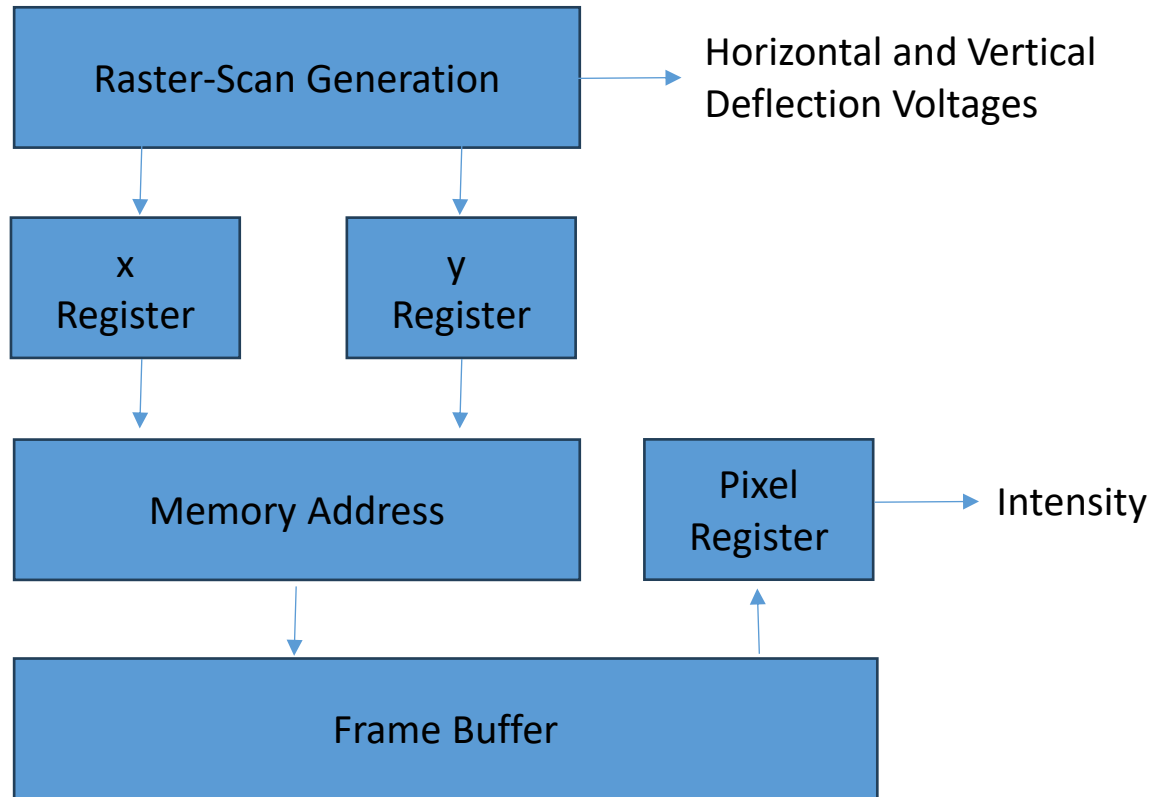


Figure: Basic video-controller refresh operations.

The Display Hardware Interface

The Video Controller (or Display Controller) is the hardware bridge between the system memory and the display device.

Refresh Management: It continuously reads data from the Frame Buffer to refresh the screen at the required rate.

Timing: Generates all essential timing signals (Horizontal and Vertical Sync).

Analog Conversion: Feeds digital data to the DACs for final output to the CRT.

Primary Goal:
Ensure continuous, synchronized data transfer to the display hardware to maintain a flicker-free image.

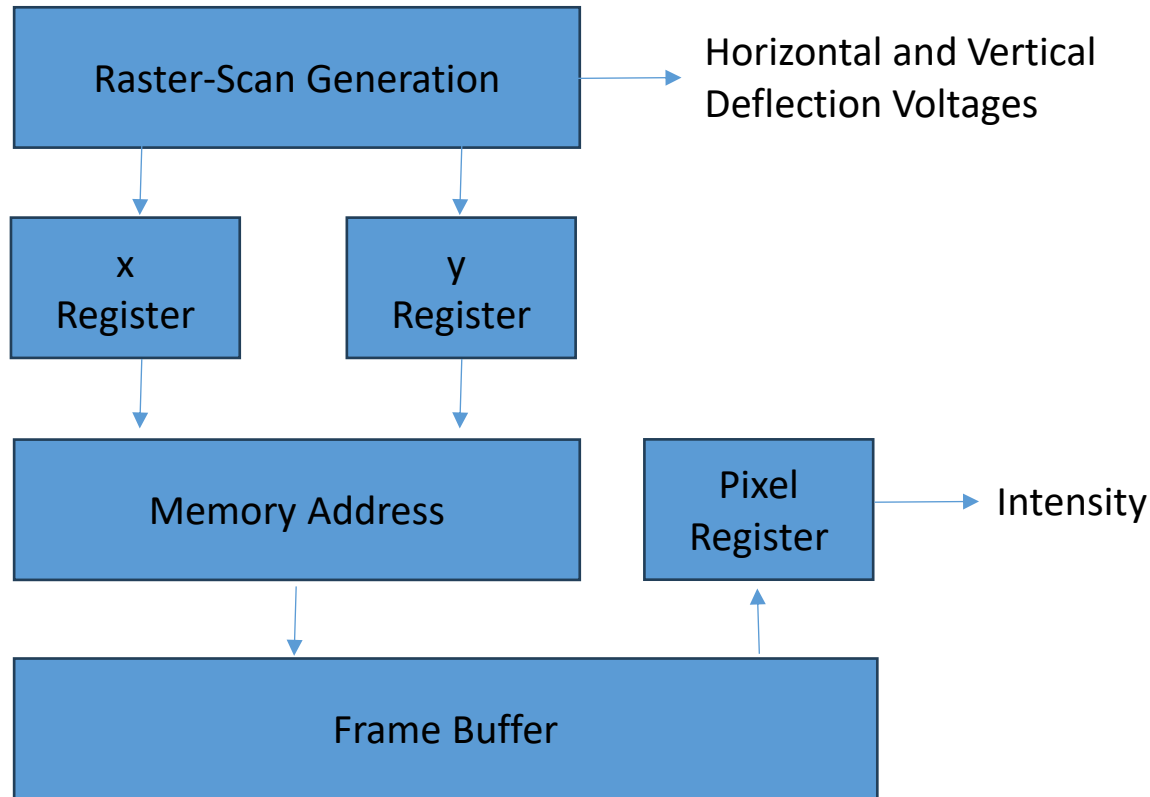


Figure: Basic video-controller refresh operations.

Key Components

Address Translator: Calculates the Frame Buffer memory address corresponding to the current screen coordinate (X, Y).

X and Y Registers: Internal counters that track the beam position during the scan process.

Control Logic: Generates the timing signals (H-Sync and V-Sync) to synchronize the electron beam's movement.

Shift Register: Used to convert the parallel pixel data read from the Frame Buffer into a serial stream for the DAC.

Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 4

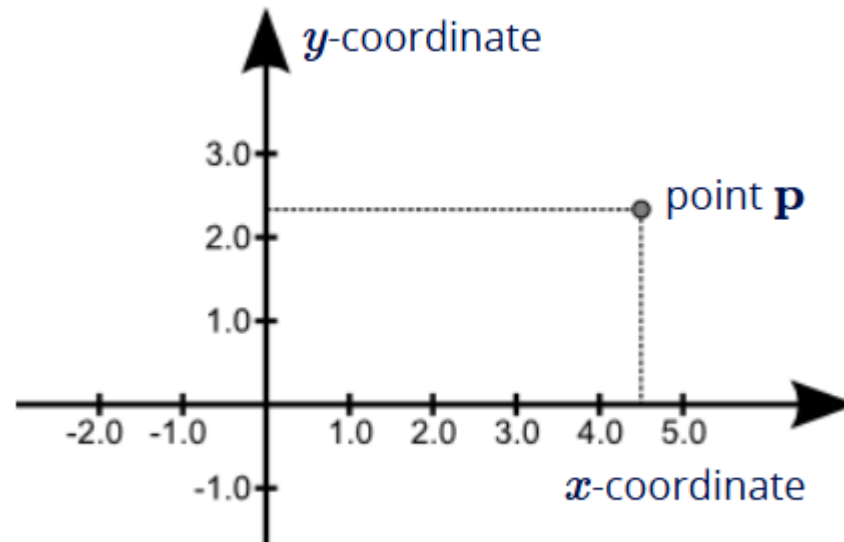
Er Sanim Kumar Khatri

CONTENT

1.4 Raster Graphics
1.5 Scan Conversion

Vector Graphics

- Mathematical Representation of Coordinates
- In computer graphics, when coordinates of 2D or 3D shapes are defined mathematically, this is ideally done by real numbers (which are represented in the computer as a floating point number)
- Example: A point in a 2D coordinate system can be defined by two real numbers $x=4.5$ and $y=2.33$



Vector Graphics

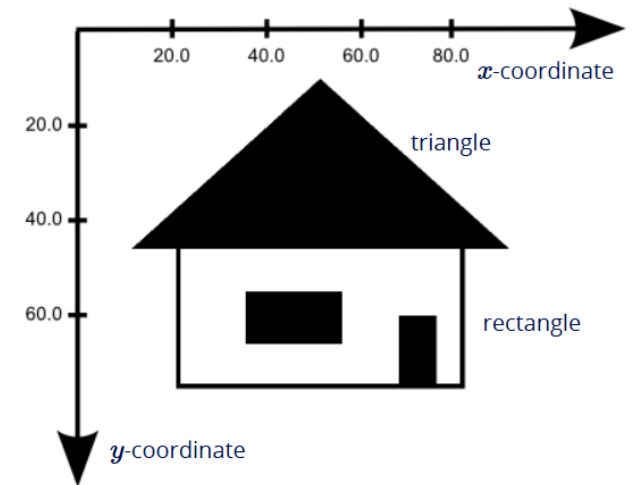
- A representation of 2D or 3D shapes with real coordinates is called Vector Graphic
- The term stems from the fact that the coordinates can be written as vectors
- For a point p in a 2D coordinate system, for example,
- $p = (x, y) = (4.5, 2.33)$
- With vector graphics, objects are described by combining basic objects (such as line, rectangle, circle, triangle, polygon)
- A representation as a vector graphic has the advantage that the objects can be arbitrarily scaled
- For example, an analog plotter can approach the coordinates accurately and thus draw a very accurate picture of the shapes



World map



Detail magnification



Vector Graphics

To create 2D vector graphics, there are many free and commercial programs:

- Inkscape
- Xfig
- OpenOffice Draw
- Adobe Illustrator
- Corel Draw
- Microsoft PowerPoint

Storing Vector Graphics

Popular file formats for storing 2D vector graphics are:

- Scalable Vector Graphics (SVG)
- PostScript (PS)
- Portable Document Format (PDF)
- Windows Metafile (WMF)

Raster Graphics

- Today's screens provide individual pixels (Pixel="Picture Elements"), which are arranged on a fixed grid
- Printers with linear paper feed (no plotters) apply their color also on a discrete dot matrix
- Raster images have a color value for each pixel of their grid. They are therefore fully defined by:
 - Image width: Number of pixels in the x direction
 - Image Height: number of pixels in the y-direction
 - and a data field with (width x height) color value
- Digital images are typical examples for raster graphic
- For example, image data of real objects are recorded with a digital camera with photosensitive pixel areas
 - 1280×1024 (1,3 megapixels)
 - 2560×2048 (5,2 megapixels)
 - 5120×4096 (21 megapixels)
- Typical conventions for raster graphics:
 - Counting of pixels starts at 0
 - x-coordinate axis to the right
 - y-coordinate axis downwards

Raster Graphics

Advantages of Raster Graphics

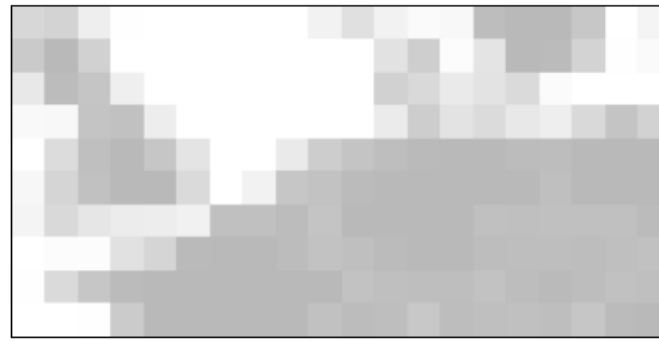
- Well suited for complex scenes with irregular shapes or many colors
- Memory requirement is independent of scene complexity
- Direct display on today's screens

Disadvantages of Raster Graphics

- When displaying very simple objects memory requirements are higher compared to a vector graphic representation
- At large magnification individual pixels become visible
- This effect is also called jaggies or aliasing



Detail magnification vector graphics



Detail magnification raster graphics

Raster Graphics

There are many free and commercial image editing programs:

- Gimp
- Microsoft Paint
- Adobe Photoshop
- Corel Photo-Paint

Storing Raster Graphics

Popular file formats for raster graphics are:

- PNG (Portable Network Graphics), lossless compression
- JPEG, smaller files because of lossy compression
- TIFF (Tagged Image File Format), typically lossless compression, high color depth (up to 32 bit)
- TGA (Targa Image File), uncompressed or lossless compression
- GIF (Graphics Interchange Format), lossless compression, allows animation sequences

Raster Principle

- **The Grid:** Unlike vector paths, raster images are composed of a fixed grid of values.
- **Display Technology:** Almost all modern displays (LCD, OLED) are inherently raster devices.
- **Finite Resolution:** You cannot zoom in infinitely; eventually, you see the atomic units of the image.
- **Photographic Realism:** Raster is the dominant format for photography due to its ability to handle complex color gradients.

Continuous vs. Discrete

The physical world is **continuous**. Light intensity and spatial position can vary infinitesimally.

Computers are **discrete**. They operate in finite steps. To represent the world, we must map infinite possibilities into finite "buckets".

This translation process is the foundation of all digital signal processing.

The Mapping Process

1. Sampling

Discretizing Space (Where?)

We take measurements of the image at regular intervals on a grid.

This determines the **spatial resolution** (width x height).

"How many boxes do we have?"

2. Quantization

Discretizing Value (What?)

We assign a specific, finite integer value to the intensity or color of each sample.

This determines the **color depth** (bits per pixel).

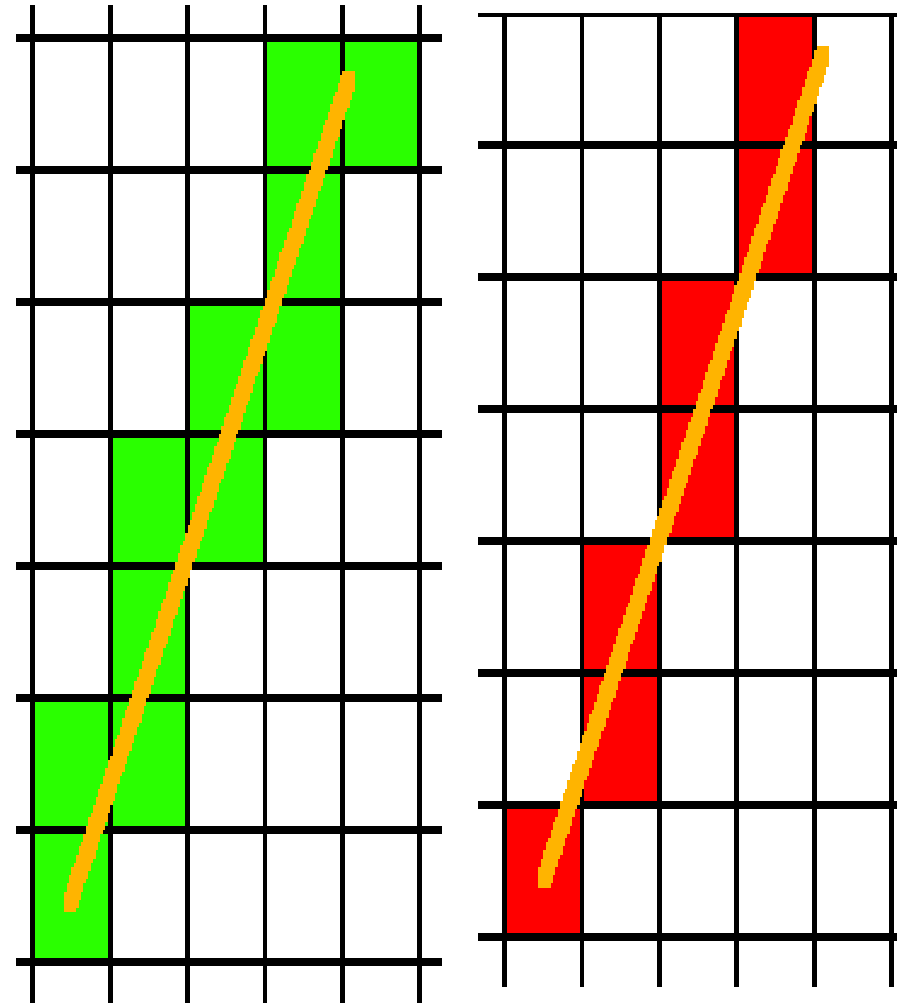
"Which color goes in the box?"

Line Digitization

The Decision Rule

When drawing a line from (x_1, y_1) to (x_2, y_2) on grid paper:

- Calculate the ideal mathematical path.
- At each column increment (x), check the vertical position (y).
- **Round to the nearest integer.**



Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 5

Er Sanim Kumar Khatri

CONTENT

1.5 Scan Conversion

Line Drawing – DDA Algorithm

Line Drawing Algorithm

The slope-intercept equation of a straight line is:

$$y = mx + b \text{(1)}$$

Where

m = slope of line

b = y-intercept

For any two points (x_1, y_1) and (x_2, y_2) .

$$m = (y_2 - y_1) / (x_2 - x_1) = dy/dx \text{(2)}$$

$$b = y_1 - mx_1 \text{(3)}$$

For any point (x_k, y_k)

$$y_k = mx_k + b \text{(4)}$$

At point x_{k+1}, y_{k+1}

$$y_{k+1} = mx_{k+1} + b \dots\dots\dots(5)$$

Subtracting these equations

$$y_{k+1} - y_k = m (x_{k+1} - x_k)$$

$(y_{k+1} - y_k)$ is the increment in y as corresponding increment in x .

For given x interval Δx along a line, we can compute the corresponding y interval Δy from the following equation:

$$\Delta y = m \Delta x \dots\dots\dots(6)$$

Similarly, we can obtain the x interval Δx corresponding to specified Δy as:

$$\Delta x = \Delta y / m \dots\dots\dots(7)$$

Case I: For lines with slope magnitudes $m < 1$, Δx can be set proportional to a unit horizontal deflection voltage and the corresponding vertical deflection is then set proportional to Δy as calculated from equation 6.

Case II: For lines whose slopes have magnitude $m > 1$, Δy can be set proportional to a unit vertical deflection voltage and the corresponding horizontal deflection voltage set proportional to Δx , calculated from equation 7.

Case III: For lines with $m=1$, $\Delta x = \Delta y$ and the horizontal and vertical deflection voltages are equal.

On raster systems, we must sample a line at discrete positions and determine the nearest pixel to the line at each sampled position.

DDA Line Drawing Algorithm

DDA algorithm is a scan conversion line algorithm based on calculation either Δx or Δy based on relation

$$\Delta y = m \Delta x \dots\dots\dots(i)$$

$$\Delta x = \Delta y / m \dots\dots\dots(ii)$$

We sample the line at unit interval in one coordinate and determine corresponding integer values nearest the path of the other co-ordinate.

Different cases:

Case 1: Line with positive slope and magnitude less than or equal to 1 ($|m| \leq 1$)

Case 2: Line with positive slope and magnitude greater than to 1 ($|m| > 1$)

Case 3: Line with negative slope and magnitude less than or equal to 1 ($|m| \leq 1$)

Case 4: Line with negative slope and magnitude greater than to 1 ($|m| > 1$)

Digital Differential Analyzer Algorithm (DDA)

Line with positive slope

Case I: If slope (m) ≤ 1 , then sample at unit x intervals ($\Delta x = 1$) and compute each successive y value because the increment in x is more than increment in y .

Set, $\Delta x = 1$

Then, $\Delta y = m$

$$x_{k+1} = x_k + \Delta x = x_k + 1$$

Then,

$$y_{k+1} = y_k + m$$

Digital Differential Analyzer Algorithm (DDA)

Line with positive slope

Case II: If slope (m) > 1 , then sample at unit y intervals ($\Delta y = 1$) and compute each successive y value because the increment in y is more than increment in x .

Set, $\Delta y = 1$

Then, $\Delta x = 1 / m$

$$x_{k+1} = x_k + \Delta x = x_k + 1/m$$

Then,

$$y_{k+1} = y_k + 1$$

These cases assumes that lines are to be drawn from left to right.
If the lines drawing process is reversed, the equation are changed.

If process is reversed

Case I: If $m \leq 1$, $\Delta x = -1$, then, $\Delta y = -m$

$$x_{k+1} = x_k - 1$$

$$y_{k+1} = y_k - m$$

Case II: If $m > 1$, $\Delta y = -1$, then, $\Delta x = -1 / m$

$$x_{k+1} = x_k - 1/m$$

$$y_{k+1} = y_k - 1$$

Digital Differential Analyzer Algorithm (DDA)

Line with negative slope

Case I: If slope $|m| \leq 1$, then assume the end point is left.

Set, $\Delta x = 1$

Then, $\Delta y = m$ (m is negative)

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$

If processed from left to right:

$\Delta x = -1$, then, $\Delta y = -m$

$$x_{k+1} = x_k - 1$$

$$y_{k+1} = y_k - m$$

Line with negative slope

Case II: If slope $|m| > 1$, then assume the end point is left.

Set, $\Delta y = 1$

Then, $\Delta x = 1 / m$ (m is negative)

$$x_{k+1} = x_k + 1/m$$

$$y_{k+1} = y_k + 1$$

If processed from left to right:

$\Delta y = -1$, then, $\Delta x = -1 / m$

$$x_{k+1} = x_k - 1/m$$

$$y_{k+1} = y_k - 1$$

DDA Algorithm:

1. Start
2. Declare $x_1, y_1, x_2, y_2, dx, step$ as integer variable and x, y, x_{inc}, y_{inc} as floating point.
3. Enter value of x_1, y_1, x_2, y_2
4. Calculate $dx = x_2 - x_1$
5. Calculate $dy = y_2 - y_1$
6. If $|dx| > |dy|$
Then, $step = |dx|$
Else, $step = |dy|$
7. $x_{inc} = dx / step$
 $y_{inc} = dy / step$
assign $x = x_1$
assign $y = y_1$
8. Set pixel (x_1, y_1)

Digital Differential Analyzer Algorithm (DDA)

9. $x = x + x_{inc}$
 $y = y + y_{inc}$
 Set pixel (x, y) (rounded value)
10. Repeat until $x = x_2$ $y = y_2$
11. End

Advantage:

1. It is faster method than direct line drawing equation $y = mx + c$ for pixel calculation, as it eliminates multiplication

Disadvantage:

1. The floating point addition is still needed in determining each successive point which is time consuming. The value of slope 'm' is usually stored in floating point number. Can be round off error.
2. The line will move away from the true line path, especially when it is long due to successive round off error.

Digital Differential Analyzer Algorithm (DDA)

Q. Calculate the points between the starting point (5, 4) and ending point (12, 7).

Digital Differential Analyzer Algorithm (DDA)

	0	1	2	3	4	5	6	7	8	9	10	11	12
0													
1													
2													
3													
4													
5													
6													
7													
8													
9													
10													
11													

Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 6

Er Sanim Kumar Khatri

CONTENT

1.5 Scan Conversion **Bresenham's Algorithm**

Bresenham's Algorithm

For positive slope and $|m| < 1$

- Pixel positions are determined by sampling at unit x intervals.
- Starting from left end position (x_0, y_0) of a given line, we step to each successive column (x-position) and plot the pixel whose scan line y value is closed to the line path.

Assuming the pixel at (x_k, y_k) to be displayed is determined, we next to decide which pixel to plot in column x_{k+1} , our choices are the pixels at positions:

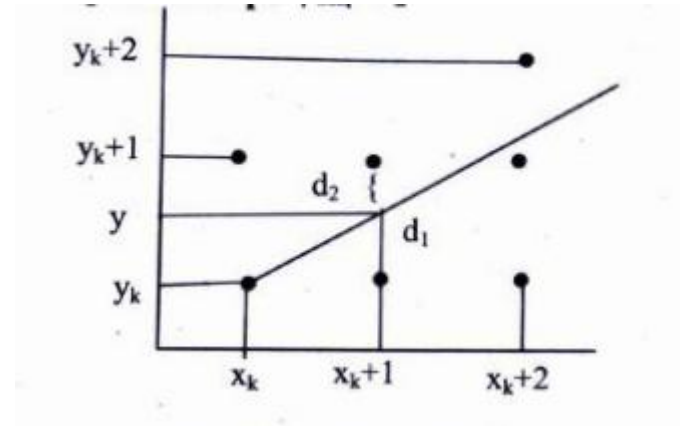
x_{k+1}, y_k and x_{k+1}, y_{k+1}

At sampling position x_{k+1} , we label vertical pixel separations from the mathematical line path d_1 and d_2 .

The y-co-ordinate on the mathematical at pixel column position x_{k+1} is calculation.

As,

$$y = m(x_{k+1}) + b_1 \text{ -----(1)}$$



$$d_1 = y - y_k$$

$$d_1 = m(x_{k+1}) + b_1 - y_k \text{ -----}(2)$$

Then,

$$d_2 = y_{k+1} - y$$

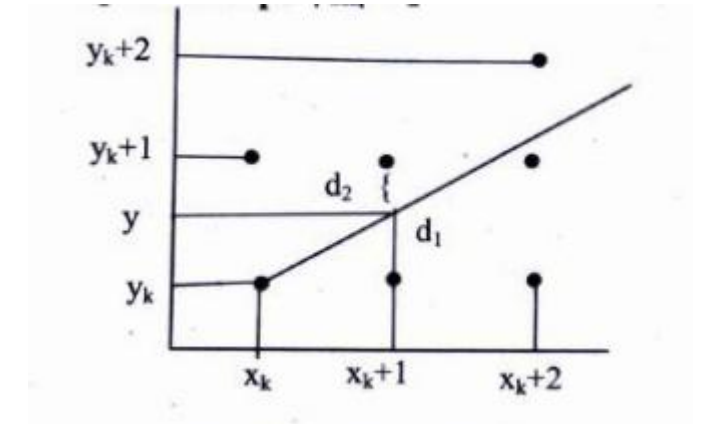
$$d_2 = y_{k+1} - m(x_{k+1}) - b \text{ -----}(3)$$

Now,

$$d_1 - d_2 = m(x_k + 1) + b - y_k - y_{k+1} + m(x_k + 1) + b$$

$$= 2m(x_k + 1) - 2y_k + 2b - 1$$

$$= (2\Delta y / \Delta x)(x_k + 1) - 2y_k + 2b - 1$$



Define decision parameter:

$$p_k = \Delta x(d_1 - d_2)$$

So,

$$p_k = \Delta x(d_1 - d_2)$$

$$= 2\Delta y(x_k + 1) - 2\Delta x y_k + 2\Delta x b - \Delta x$$

$$= 2\Delta y x_k + 2\Delta y - 2\Delta x y_k + 2\Delta x b - \Delta x$$

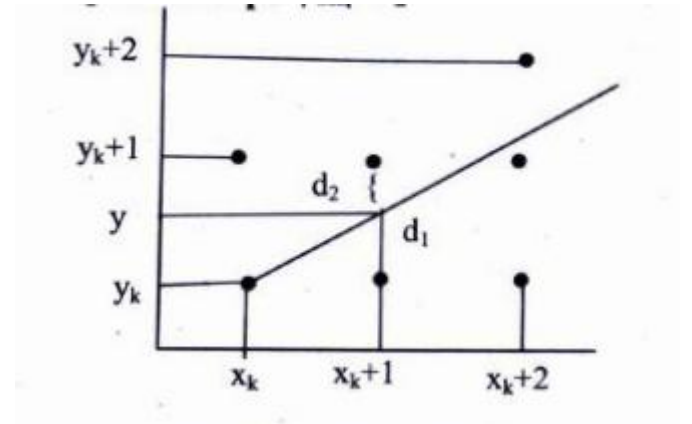
$$= 2\Delta y x_k - 2\Delta x y_k + 2\Delta y + \Delta x(2b - 1)$$

$$= 2\Delta y x_k - 2\Delta x y_k + c$$

Where:

$$c = 2\Delta y + \Delta x(2b - 1)$$

Sign of p_k is same as sign of $(d_1 - d_2)$ since $\Delta x > 0$.



Next, compute p_{k+1} :

$$p_{k+1} = 2\Delta y x_{k+1} - 2\Delta x y_{k+1} + c$$

Now,

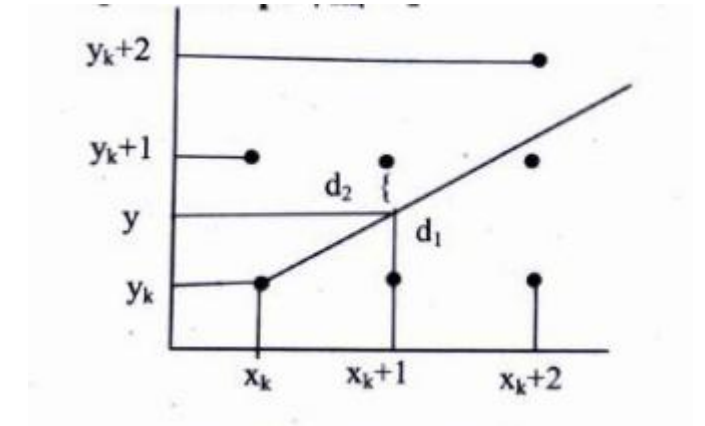
$$\begin{aligned} p_{k+1} - p_k &= (2\Delta y x_{k+1} - 2\Delta x y_{k+1} + c) - (2\Delta y x_k - 2\Delta x y_k + c) \\ &= 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k) \end{aligned}$$

Since $x_{k+1} = x_k + 1$:

$$p_{k+1} - p_k = 2\Delta y(1) - 2\Delta x(y_{k+1} - y_k)$$

$$p_{k+1} - p_k = 2\Delta y - 2\Delta x(y_{k+1} - y_k)$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k)$$



Case I

If $p_k < 0$, then $d_1 < d_2$.

This means y_k is nearer than $y_k + 1$.

So choose pixel $(x_k + 1, y_k)$.

Thus next pixel = $(x_k + 1, y_k)$

So,

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k)$$

$$p_{k+1} = p_k + 2\Delta y$$

Case II

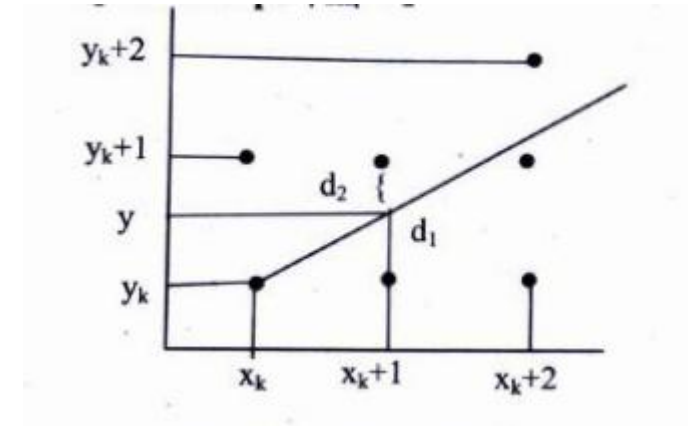
If $p_k \geq 0$, then $d_2 < d_1$.

This means $y_k + 1$ is nearer than y_k .

So choose pixel $(x_k + 1, y_k + 1)$.

Thus next pixel = $(x_k + 1, y_k + 1)$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$



Initial decision parameter (p_0)

$$p_k = 2\Delta y x_k - 2\Delta x y_k + 2\Delta y + \Delta x(2b - 1)$$

$$p_0 = 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + \Delta x(2b - 1)$$

But

$$b = y_0 - m x_0$$

$$m = \Delta y / \Delta x$$

So,

$$b = y_0 - (\Delta y / \Delta x) x_0$$

Therefore,

$$\begin{aligned} p_0 &= 2\Delta y x_0 - 2\Delta x y_0 + 2\Delta y + 2\Delta x y_0 - 2\Delta y x_0 - \Delta x \\ &= 2\Delta y - \Delta x \end{aligned}$$

Algorithm For $|m| \leq 1$

1. Start
2. Read x_a, y_a, x_b, y_b (Assume $-1 \leq m \leq 1$)
3. Load (x_a, y_a) into the frame buffer (i.e., plot the first point)
4. Calculate constants $\Delta y, \Delta x, 2\Delta y$ and $2\Delta y - 2\Delta x$
Obtain the first decision parameter $p_0 = 2\Delta y - \Delta x$
5. At each x_k along the line starting at $k = 0$ perform the following tests:
If $p_k < 0$, then the next point to plot is $(x_k + 1, y_k)$ and
$$p_{k+1} = p_k + 2\Delta y$$

Else the next point to plot is $(x_k + 1, y_k + 1)$ and
$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$
6. Repeat step iv Δx times
7. Stop

Advantages:

- It is easy to implement.
- It is fast and incremental.
- It executes fast but less faster than DDA Algorithm.
- The points generated by this algorithm are more accurate than DDA Algorithm.
- It uses fixed points only.

Disadvantages:

- Though it improves the accuracy of generated points but still the resulted line is not smooth.
- This algorithm is for the basic line drawing.
- It cannot handle diminishing jaggies.

Rasterize the points of given line end points (2, 6) and (6, 9) using BLA?

BLA

	0	1	2	3	4	5	6	7	8	9	10	11	12
0													
1													
2													
3													
4													
5													
6													
7													
8													
9													
10													
11													

Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 7

Er Sanim Kumar Khatri

CONTENT

1.5 Scan Conversion **Bresenham's Algorithm**

Bresenham's Algorithm

For positive slope and $|m| > 1$

- Pixel positions are determined by sampling at unit y intervals.

Assuming the pixel at (x_k, y_k) to be displayed is determined, we determine next pixel (x_{k+1}, y_{k+1}) as $(x_k, y_k + 1)$ and $(x_k + 1, y_k + 1)$

At sampling position $y_k + 1$, we label horizontal pixel separations from the mathematical line path d_1 and d_2 . The x-co-ordinate on the mathematical at pixel column position $y_k + 1$ is calculation.

As,

$$y_k + 1 = mx + b$$

$$x = (y_k + 1 - b) / m \text{ -----(1)}$$

$$d1 = x - x_k$$

$$d1 = (y_k + 1 - b) / m - x_k \text{ -----}(2)$$

Then,

$$d_2 = (x_k + 1) - x$$

$$d_2 = x_k + 1 - (y_k + 1 - b) / m \text{ -----}(3)$$

Now,

$$\begin{aligned} d1 - d2 &= (y_k + 1 - b) / m - x_k - (x_k + 1 - (y_k + 1 - b) / m) \\ &= (y_k + 1 - b) / m - x_k - x_k - 1 + (y_k + 1 - b) / m \\ &= 2((y_k + 1 - b) / m) - 2x_k - 1 \\ &= (2\Delta x / \Delta y)(y_k + 1 - b) - 2x_k - 1 \end{aligned}$$

Define decision parameter:

$$p_k = \Delta y(d_1 - d_2)$$

So,

$$p_k = \Delta y(d_1 - d_2)$$

$$= 2\Delta x(y_k + 1 - b) - 2\Delta y x_k - \Delta y$$

$$= 2\Delta x y_k + 2\Delta x - 2\Delta x b - 2\Delta y x_k - \Delta y$$

$$= 2\Delta x y_k - 2\Delta y x_k + 2\Delta x(1 - b) - \Delta y$$

$$= 2\Delta x y_k - 2\Delta y x_k + c$$

Where:

$$c = 2\Delta x(1 - b) - \Delta y$$

Sign of p_k is same as sign of $(d_1 - d_2)$ since $\Delta y > 0$.

Next, compute p_{k+1} :

$$p_{k+1} = 2\Delta x y_{k+1} - 2\Delta y x_{k+1} + c$$

Now,

$$p_{k+1} - p_k = (2\Delta x y_{k+1} - 2\Delta y x_{k+1} + c)$$

$$- (2\Delta x y_k - 2\Delta y x_k + c)$$

$$= 2\Delta x(y_{k+1} - y_k) - 2\Delta y(x_{k+1} - x_k)$$

Since $y_{k+1} = y_k + 1$:

$$p_{k+1} - p_k = 2\Delta x(1) - 2\Delta y(x_{k+1} - x_k)$$

$$p_{k+1} - p_k = 2\Delta x - 2\Delta y(x_{k+1} - x_k)$$

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y(x_{k+1} - x_k)$$

Case I

If $p_k < 0$, then $d_1 < d_2$.

This means x_k is nearer than $x_k + 1$.

So choose pixel $(x_k, y_k + 1)$.

Thus next pixel = $(x_k, y_k + 1)$

So,

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y(x_{k+1} - x_k)$$

$$p_{k+1} = p_k + 2\Delta x$$

Case II

If $p_k \geq 0$, then $d_2 < d_1$.

This means $x_k + 1$ is nearer than x_k .

So choose pixel $(x_k + 1, y_k + 1)$.

Thus next pixel = $(x_k + 1, y_k + 1)$

$$p_{k+1} = p_k + 2\Delta x - 2\Delta y$$

Initial decision parameter (p_0)

$$p_k = 2\Delta x y_k - 2\Delta y x_k + 2\Delta x(1-b) - \Delta y$$

$$p_0 = 2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x(1-b) - \Delta y$$

But

$$b = y_0 - m x_0$$

$$m = \Delta y / \Delta x$$

So,

$$b = y_0 - (\Delta y / \Delta x) x_0$$

Therefore,

$$p_0 = 2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x - 2\Delta x b - \Delta y$$

$$= 2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x - 2\Delta x(y_0 - (\Delta y / \Delta x) x_0) - \Delta y$$

$$= 2\Delta x y_0 - 2\Delta y x_0 + 2\Delta x - 2\Delta x y_0 + 2\Delta y x_0 - \Delta y$$

$$= 2\Delta x - \Delta y$$

Algorithm For $|m| \geq 1$

1. Start
2. Read x_a, y_a, x_b, y_b
3. Load (x_a, y_a) into the frame buffer (i.e., plot the first point)
4. Calculate constants $\Delta y, \Delta x, 2\Delta x$ and $2\Delta x - 2\Delta y$
Obtain the first decision parameter $p_0 = 2\Delta x - \Delta y$
5. At each y_k along the line starting at $k = 0$ perform the following tests:
If $p_k < 0$, then the next point to plot is $(x_k, y_k + 1)$ and
$$p_{k+1} = p_k + 2\Delta x$$

Else the next point to plot is $(x_k + 1, y_k + 1)$ and
$$p_{k+1} = p_k + 2\Delta x - 2\Delta y$$
6. Repeat step iv Δy times
7. Stop

Advantages:

- It is easy to implement.
- It is fast and incremental.
- It executes fast but less faster than DDA Algorithm.
- The points generated by this algorithm are more accurate than DDA Algorithm.
- It uses fixed points only.

Disadvantages:

- Though it improves the accuracy of generated points but still the resulted line is not smooth.
- This algorithm is for the basic line drawing.
- It cannot handle diminishing jaggies.

Rasterize the points of given line end points (2, 4) and (6, 9) using BLA?

BLA

	0	1	2	3	4	5	6	7	8	9	10	11	12
0													
1													
2													
3													
4													
5													
6													
7													
8													
9													
10													
11													

Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 8

Er Sanim Kumar Khatri

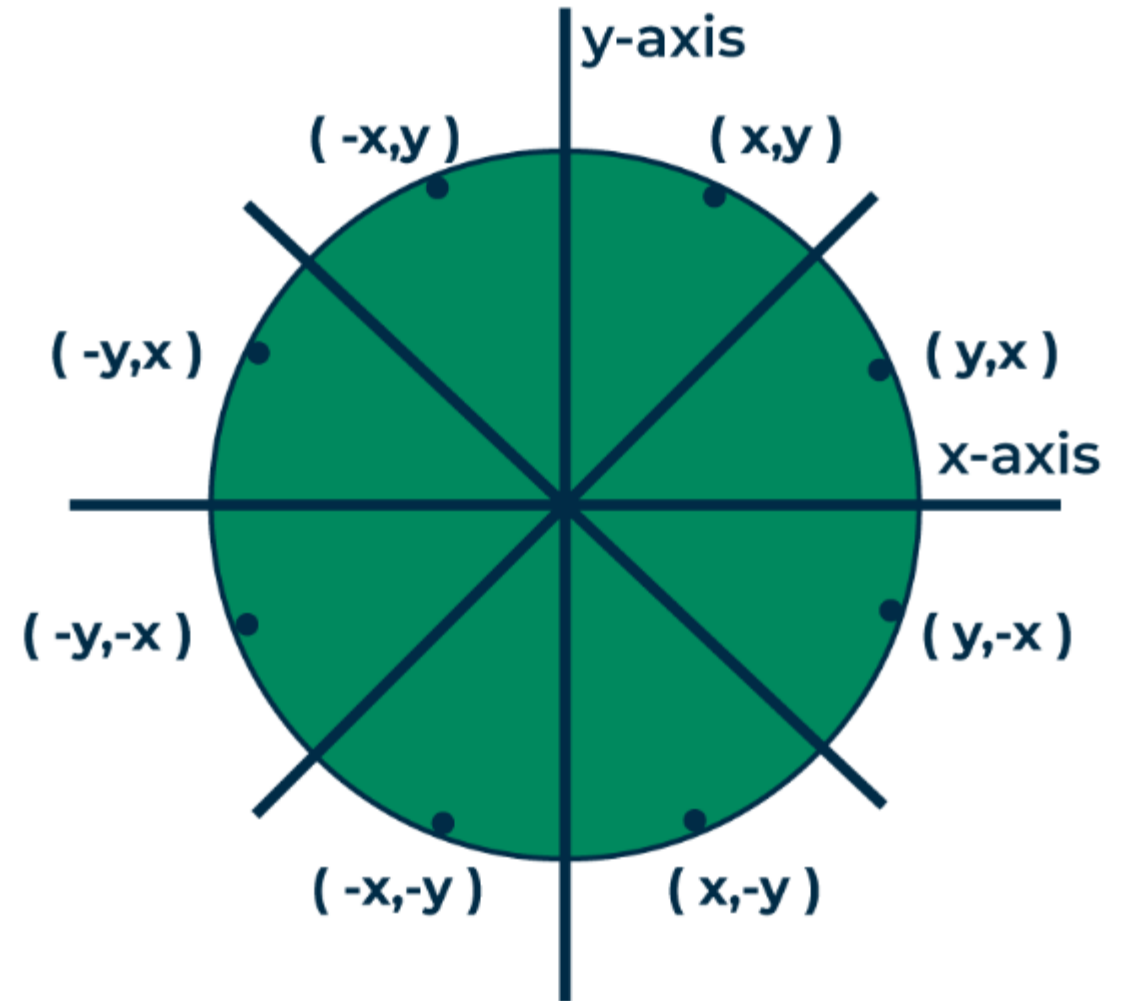
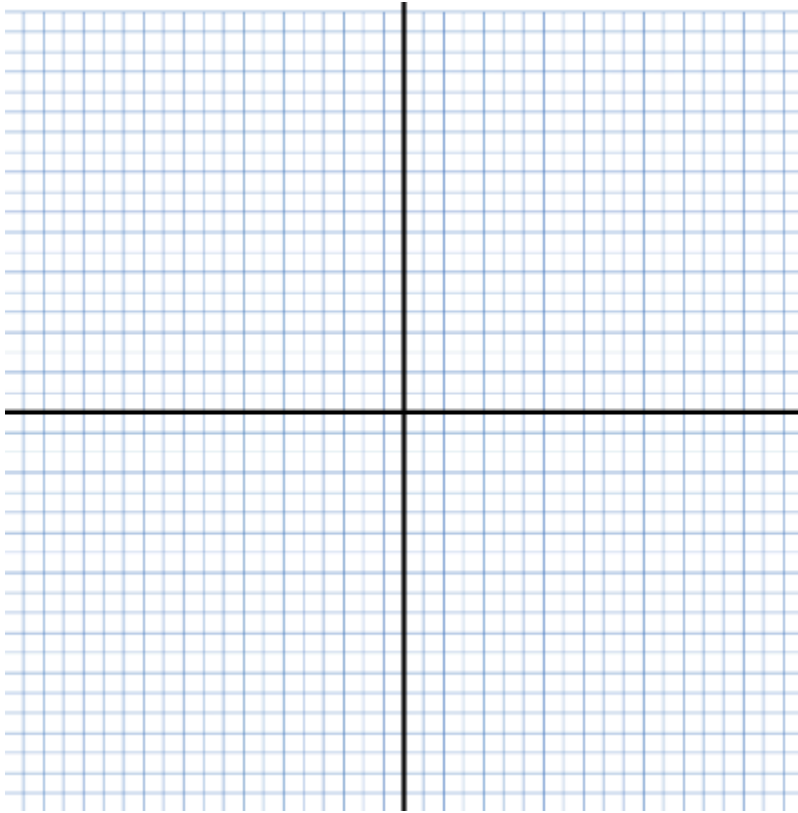
CONTENT

1.5 Scan Conversion **Circle**

Circle

Symmetry in Circle

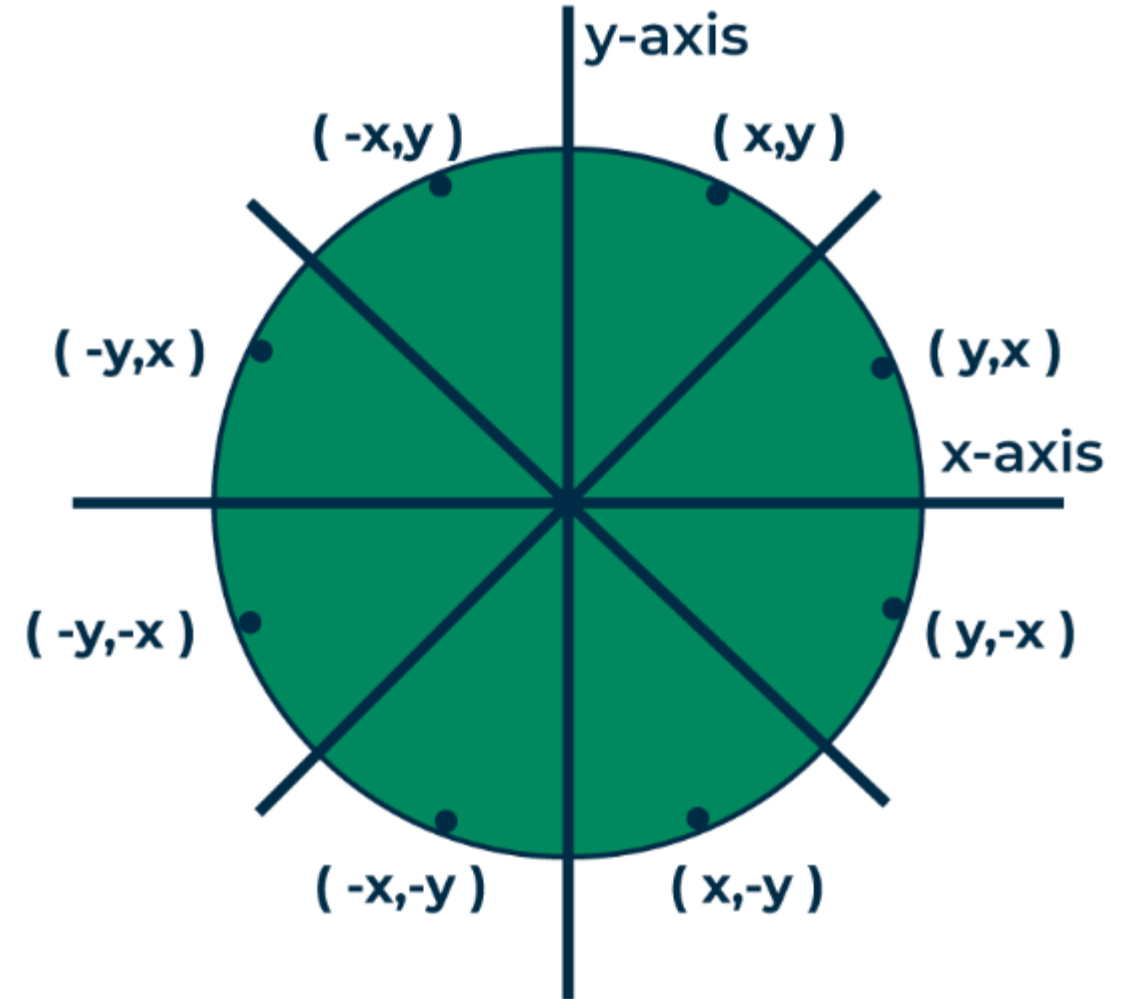
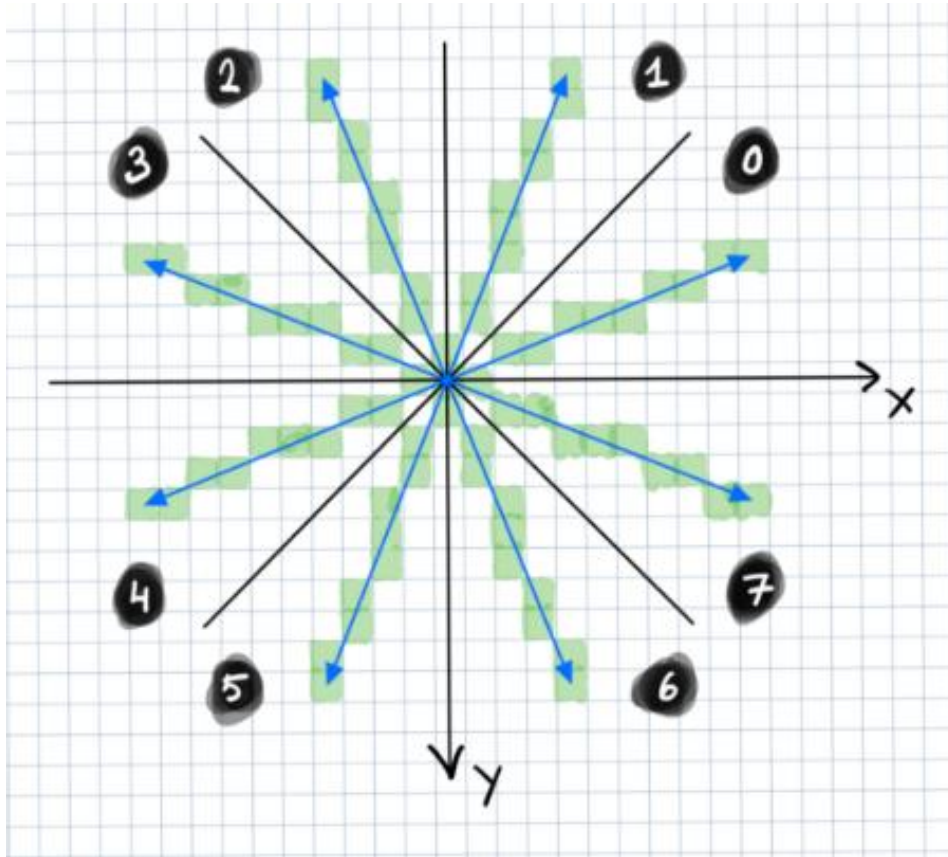
There are 4 quadrants and 8 octants.



Circle

Symmetry in Circle

There are 4 quadrants and 8 octants.



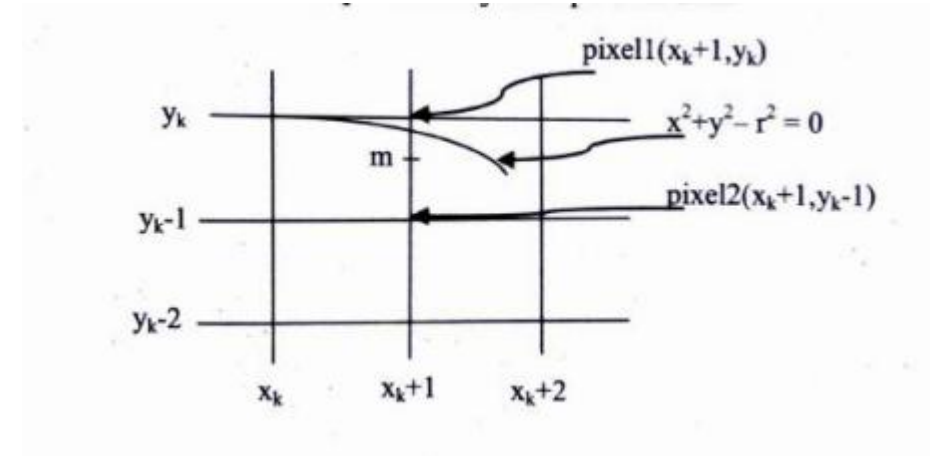
Circle

Midpoint Circle Algorithm.

The equation of a circle is given by

$$x^2 + y^2 = r^2$$

With center (0, 0) and radius r, let's define a circle function as circle $(x, y) = x^2 + y^2 - r^2$.



In figure, length between pixel 1 and pixel 2 is $(y_k - y_{k+1}) = 1$ and half the length = $1/2$

$F_{\text{circle}}(x, y) < 0$ if (x, y) is inside the circle boundary
= 0 if (x, y) is on the circle boundary
> 0 if (x, y) is outside the circle boundary

This circle function $F_{\text{circle}}(x, y)$ serves as the decision parameter

Circle

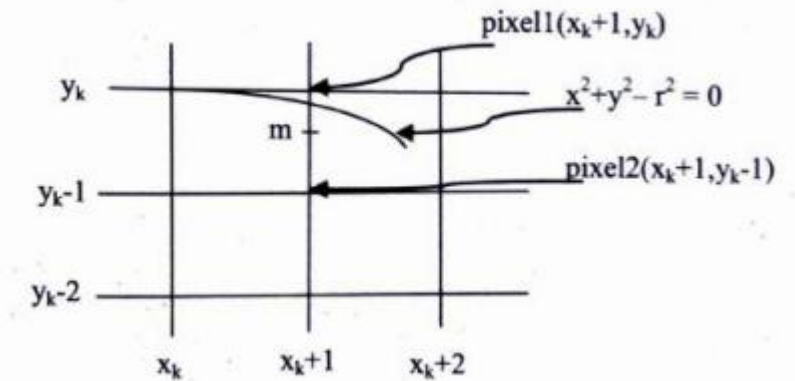
Assume (x_k, y_k) is plotted, then next point close to the circle is (x_k+1, y_k) or (x_k+1, y_k-1)

The decision parameter is the circle function evaluated at the midpoint between these two pixels.

$$\begin{aligned} P_k &= F_{\text{circle}}(x_k + 1, y_k - \frac{1}{2}) \\ &= (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2 \text{ ----- (i)} \end{aligned}$$

At the next sampling position $(x_{k+1} + 1 = x_k + 2)$, the decision parameter is evaluated as

$$\begin{aligned} P_{k+1} &= F_{\text{circle}}(x_{k+1} + 1, y_{k+1} - \frac{1}{2}) \\ &= [(x_k + 1) + 1]^2 + (y_{k+1} - \frac{1}{2})^2 - r^2 \text{ -----(ii)} \end{aligned}$$



Circle

Now subtracting eq (i) and (ii),

$$\begin{aligned} P_{k+1} - P_k &= [(x_k + 1) + 1]^2 + (y_{k+1} - \frac{1}{2})^2 - r^2 - ((x_{k+1})^2 + (y_{k - \frac{1}{2}})^2 - r^2) \\ &= (x_k + 1)^2 + 2(x_k + 1) + 1 + (y_{k+1} - \frac{1}{2})^2 - r^2 - (x_{k+1})^2 - \\ &\quad (y_{k - \frac{1}{2}})^2 + r^2 \\ P_{k+1} &= P_k + 2(x_k + 1) + (y_{k+1} - \frac{1}{2})^2 - (y_{k - \frac{1}{2}})^2 + 1 \text{ -----(iii)} \end{aligned}$$

where y_{k+1} is either y_k or $y_k - 1$ depending on the sign of P_k .

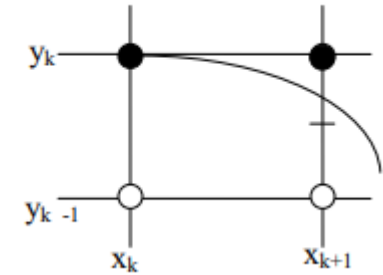
Case 1:

If $P_k < 0$ then the mid point is inside the circle, so pixel on scanline

' y_k ' is closer to the circle boundary and $y_{k+1} = y_k$

From equation (iii)

$$\begin{aligned} \text{or } P_{k+1} &= P_k + 2(x_k + 1) + (y_k - \frac{1}{2})^2 - (y_k - \frac{1}{2})^2 + 1 \\ &= P_k + 2(x_k + 1) + 1 \\ &= P_k + 2(x_{k+1}) + 1 \end{aligned}$$

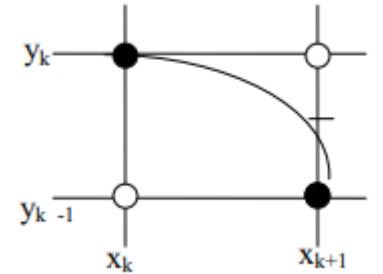


Circle

Case 2:

If $P_k \geq 0$ then the mid point is outside or on the boundary of the circle, so we select the pixel on scan line ' $y_k - 1$ ' then $y_{k+1} = y_k - 1$ i.e. from equation (iii)

$$\begin{aligned}P_{k+1} &= P_k + 2(x_k + 1) + (y_k - 1 - \frac{1}{2})^2 - (y_k - \frac{1}{2})^2 + 1 \\&= P_k + 2(x_k + 1) + (y_k - \frac{1}{2})^2 - 2(y_k - \frac{1}{2}) + 1 - (y_k - \frac{1}{2})^2 + 1 \\&= P_k + 2(x_k + 1) - 2y_k + 1 + 1 + 1 \\&= P_k + 2(x_k + 1) - 2(y_k - 1) + 1 \\&= P_k + 2(x_{k+1}) - 2(y_{k+1}) + 1\end{aligned}$$



Circle

The initial decision parameter P_0 is obtained by evaluating the circle function at the start position $(x_0, y_0) = (0, r)$.

Next pixel to plot is either $(1, r)$ or $(1, r - 1)$ So, midpoint coordinate position is $(1, r - \frac{1}{2})$

$$F_{\text{circle}}(1, r - \frac{1}{2}) = 1 + (r - \frac{1}{2})^2 - r^2$$

$$\text{Thus, } P_0 = 5/4 - r$$

If the radius 'r' is specified as an integer, we can simply round P_0 to $P_0 = 1 - r$ (for 'r' an integer)

Circle

Mid-point circle algorithm

Step 1. Start

Step 2. Declare variables $X_c, Y_c, r, X_0, Y_0, P_0, P_k, P_{k+1}$.

Step 3. Read values of X_c, Y_c, r .

Step 4. Initialize the X_0 and y_0 i.e., set the co-ordinates for the first point on the circumference of the circle centered at origin as $X_0=0$

$Y_0=r$

Step 5. Calculate initial value of decision parameter

For Step 6. At each x position, starting from $k = 0$

If $p_k < 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_{k+1} = P_k + 2X_{k+1} + 1$$

else

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k - 1$$

$$P_{k+1} = P_k + 2X_{k+1} - 2Y_{k+1} + 1$$

Circle

Step 7. Determine the symmetry in other seven octants.

Step 8. Move each calculated pixel position (x, y) onto the circular path centered on (X_c, Y_c)

Step 9. Plot the co-ordinates values

$x = x + x_c$

$y = y + y_c$

Step 10. Repeat steps 6 to 9 until $x \geq y$

Step 11. Stop

3. Determine the raster location along the circle octant in the first quadrant for a circle with radius 10.

Solution:

$$r=10$$

Initial decision parameter,

$$p_0=1-r$$

$$=1-10$$

$$=-9$$

$$(x_0, y_0) = (0, 10)$$

$$2x_0=0, 2y_0=20$$

k	p_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$
0	-9	(1, 10)	2	20
1	-6	(2, 10)	4	20
2	-1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	-3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14

Thank you

COMPUTER GRAPHICS (CAC305)

LECTURE 9

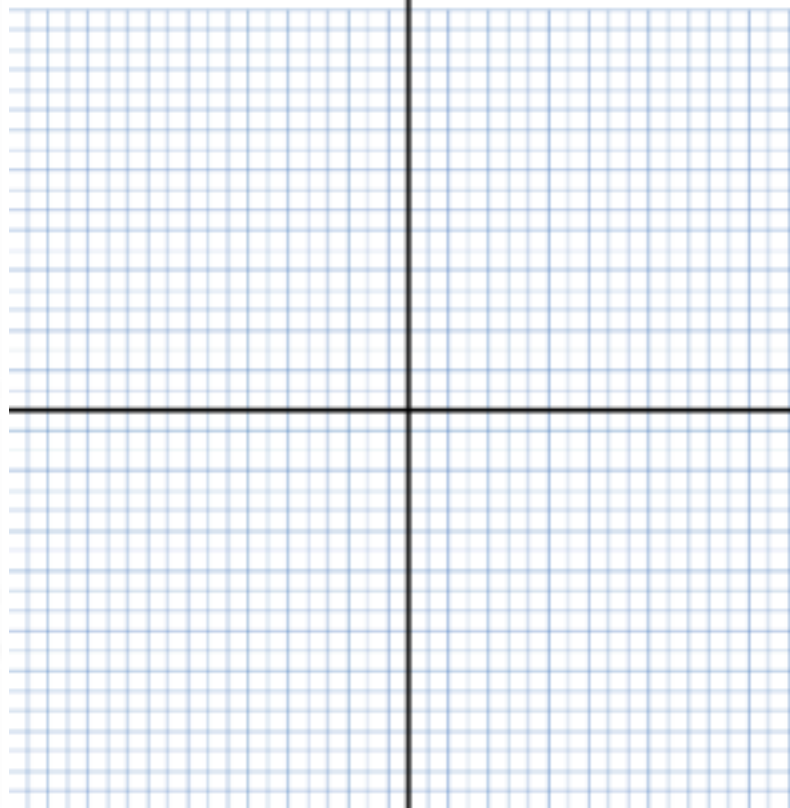
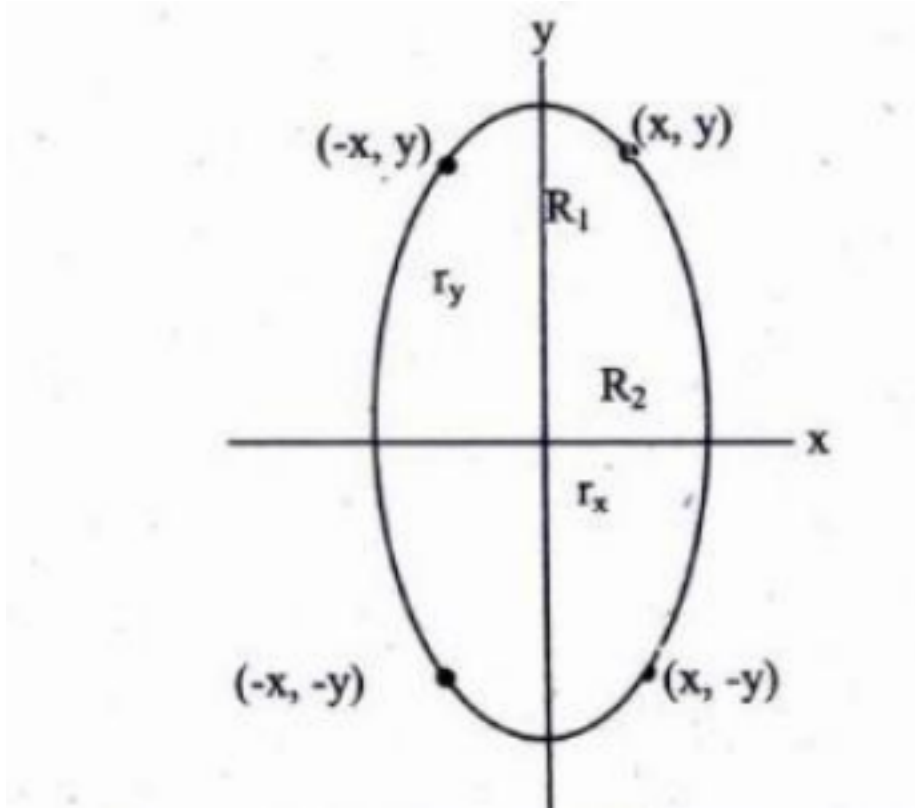
Er Sanim Kumar Khatri

1.5 Scan Conversion **Ellipse** **CONTENT**

Ellipse

Symmetry in Ellipse

There are 4 quadrants



Ellipse

Midpoint Ellipse Algorithm.

The equation of an ellipse is given by

$$x^2/r_x^2 + y^2/r_y^2 = 1$$

$$F_{\text{ellipse}}(x,y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$$

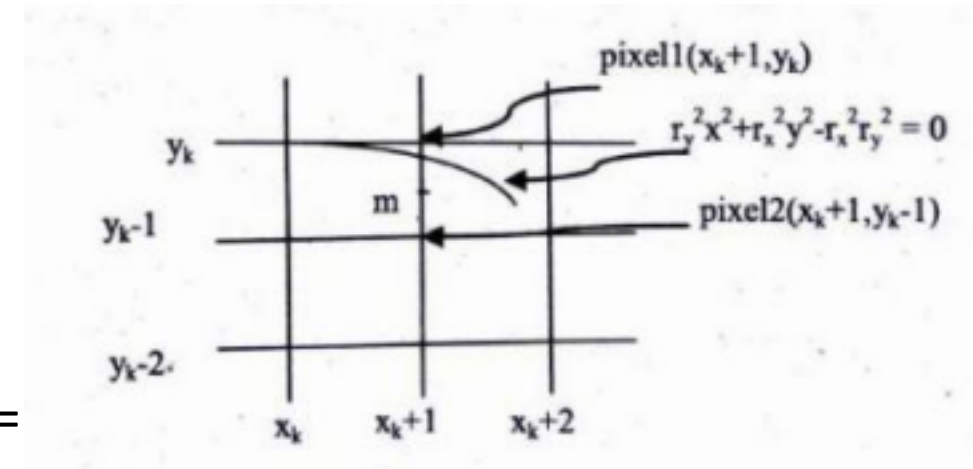
In figure, length between pixel 1 and pixel 2 is $(y_k - y_{k+1}) = 1$ and half the length $= 1/2$

$F_{\text{ellipse}}(x,y) < 0$ if (x,y) is inside the ellipse boundary =
 0 if (x,y) is on the ellipse boundary

> 0 if (x,y) is outside the ellipse boundary

This ellipse function $F_{\text{ellipse}}(x,y)$ serves as the

decision parameter



region 1

Mid-point in

Ellipse

We select next pixel along the ellipse path according to the sign of ellipse function evaluated at the midpoint between two candidate pixels. Start at $(0, r_y)$ take unit steps in 'x' direction until we reach boundary region 1 and 2

then switch to unit steps in 'y' direction for remainder of curve in the first quadrant.

At each step test the value of the slope of the curve.

The ellipse slope is given by

$$2 r_y^2 x + 2 r_x^2 y \, dy/dx = 0$$

$$dy/dx = - 2 r_y^2 x / 2 r_x^2 y$$

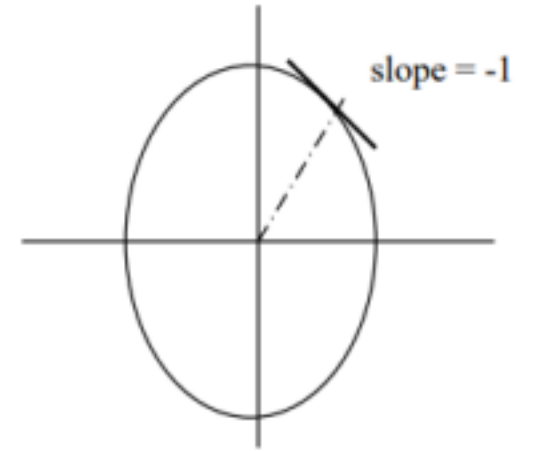
At the boundary between region 1 and 2, $dy/dx = -1$

$$\text{so, } dy/dx = -1 = - 2 r_y^2 x / 2 r_x^2 y$$

$$\text{or, } 2 r_y^2 x = 2 r_x^2 y$$

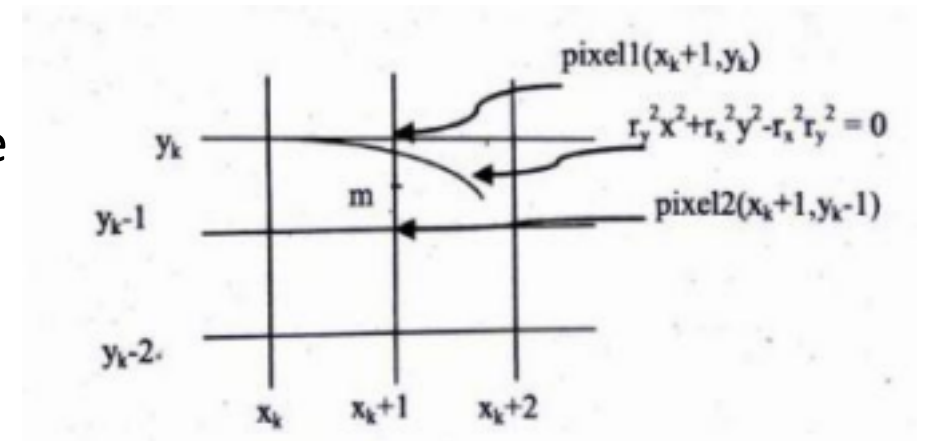
and we move out of the region 1 when $2 r_y^2 x \geq 2 r_x^2 y$

Ellipse



Assume (x_k, y_k) is plotted, then next point close to the circle is (x_k+1, y_k) or (x_k+1, y_k-1)

The decision parameter is the ellipse function evaluated at



the midpoint between these two pixels.

$$P1_k = F_{\text{ellipse}}(x_k + 1, y_k - \frac{1}{2})$$

$$= r_y^2(x_k + 1)^2 + r_x^2(y_k - \frac{1}{2})^2 - r_x^2 r_y^2 \text{-----} (i)$$

At the next sampling position ($x_{k+1} + 1 = x_k + 2$), the decision parameter is evaluated as

$$P1_{k+1} = F_{\text{ellipse}}(x_{k+1} + 1, y_{k+1} - \frac{1}{2})$$

$$= r_y^2[(x_k + 1) + 1]^2 + r_x^2(y_{k+1} - \frac{1}{2})^2 - r_x^2 r_y^2 \text{-----} (ii)$$

Ellipse

Now subtracting eq (i) and (ii),

$$P1_{k+1} - P1_k = r_y^2[(x_k + 1) + 1]^2 + r_x^2(y_{k+1} - \frac{1}{2})^2 - r_x^2 r_y^2 - (r_y^2(x_k + 1)^2 + r_x^2(y_k - \frac{1}{2})^2 - r_x^2 r_y^2)$$

$$= r_y^2[(x_k + 1)^2 + 2(x_k + 1) + 1 - (x_k + 1)^2] + r_x^2[(y_{k+1} - \frac{1}{2})^2 - (y_k - \frac{1}{2})^2] \quad P1_{k+1} = P1_k + 2r_y^2(x_k + 1)$$

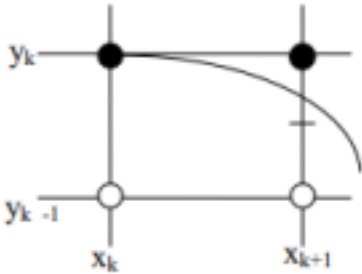
$$+ r_x^2[(y_{k+1} - \frac{1}{2})^2 - (y_k - \frac{1}{2})^2] + r_y^2 \text{-----} (iii)$$

where y_{k+1} is either y_k or $y_k - 1$ depending on the sign of P_k .

Case 1:
 If $P1_k < 0$ then the mid point is inside the ellipse, so pixel on scanline ' y_k ' is closer to the circle boundary and $y_{k+1} = y_k$

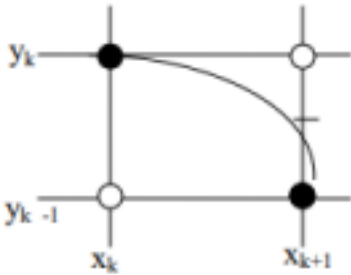
From equation (iii)
 or $P1_{k+1} = P1_k + 2r_y^2(x_k+1) + r_x^2[(y_k- \frac{1}{2})^2 - (y_k- \frac{1}{2})^2] + r_y^2$
 $= P1_k + 2r_y^2(x_k+1) + r_y^2$
 $= P1_k + 2r_y^2(x_{k+1}) + r_y^2$

Ellipse



Case 2:
 If $P_k \geq 0$ then the mid point is outside or on the boundary of the circle, so we select the pixel on scan line ' $y_k - 1$ ' then $y_{k+1} = y_k - 1$ i.e. from equation (iii)

$$\begin{aligned}
 P_{k+1} &= P1_k + 2r_y^2(x_k+1) + r_x^2[(y_k-1 - \frac{1}{2})^2 - (y_k-\frac{1}{2})^2] + r_y^2 \\
 &= P1_k + 2r_y^2(x_k+1) + r_x^2[(y_k-\frac{1}{2})^2 - 2(y_k-\frac{1}{2}) + 1 - (y_k-\frac{1}{2})^2] + r_y^2 = \\
 &P1_k + 2r_y^2(x_k+1) + r_x^2[-2y_k + 1 + 1] + r_y^2
 \end{aligned}$$



$$\begin{aligned}
&= P1_k + 2r_y^2(x_k+1) - r_x^2[2(y_k-1)] + r_y^2 \\
&= P1_k + 2r_y^2(x_{k+1}) - 2r_x^2(y_{k+1}) + r_y^2
\end{aligned}$$

Ellipse

The initial decision parameter $P1_0$ is obtained by evaluating the ellipse function at the start position $(x_0, y_0) = (0, r_y)$. Next pixel to plot is either $(1, r_y)$ or $(1, r_y - 1)$ So, midpoint coordinate position is $(1, r_y - \frac{1}{2})$

$$\begin{aligned}
F_{\text{ellipse}}(1, r_y - \frac{1}{2}) &= r_y^2(x_0 + 1)^2 + r_x^2(y_0 - \frac{1}{2})^2 - r_x^2r_y^2 \\
&= r_y^2 + r_x^2(r_y - \frac{1}{2})^2 - r_x^2r_y^2 \\
&= r_y^2 - r_x^2r_y + \frac{1}{4}r_x^2
\end{aligned}$$

Ellipse

Assuming, (x_k, y_k) has been plotted, next pixel to plot is (x_{k+1}, y_{k+1}) where x_{k+1} is either x_k or $x_k + 1$ and y_{k+1} is $y_k - 1$ i.e. we choose either $(x_k, y_k - 1)$ or $(x_{k+1}, y_k - 1)$

Region 2.

Sample at unit steps in 'y' direction, the midpoint is taken between horizontal pixels at each step now.

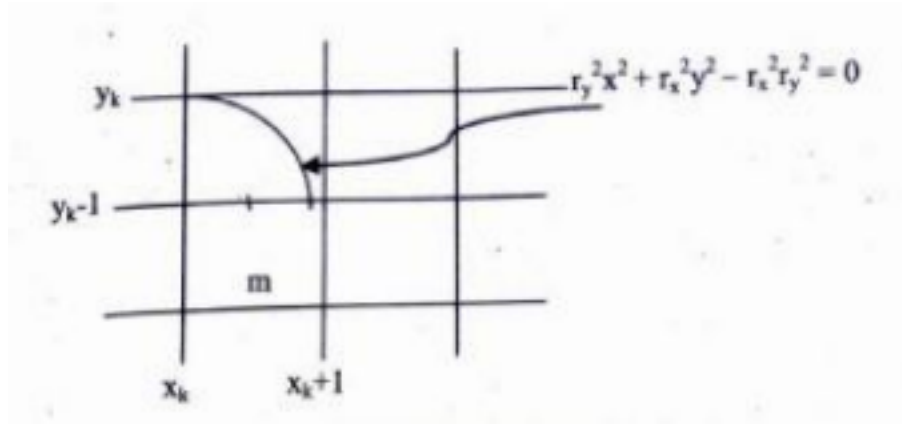
so, midpoint coordinate position is $(x_k + \frac{1}{2}, y_k - 1)$

$$F_{\text{ellipse}}(x_k + \frac{1}{2}, y_k - 1)$$

$$\text{Or, } P2_k = r_y^2(x_k + \frac{1}{2})^2 + r_x^2(y_k - 1)^2 - r_x^2 r_y^2$$

----- (iv)

region 1



Mid-point in

Ellipse

At the next sampling position, the decision parameter is evaluated as

$$F_{\text{ellipse}}(x_{k+1} + \frac{1}{2}, y_{k+1} - 1)$$

$$\text{Or, } P2_{k+1} = r_y^2(x_{k+1} + \frac{1}{2})^2 + r_x^2(y_{k+1} - 1)^2 - r_x^2 r_y^2$$

Or, $P2_{k+1} = r_y^2(x_{k+1} + \frac{1}{2})^2 + r_x^2[(y_k - 1) - 1]^2 - r_x^2 r_y^2 \text{-----} (v)$

Now subtracting eq (iv) and (v),

$$P2_{k+1} - P2_k = r_y^2(x_{k+1} + \frac{1}{2})^2 + r_x^2[(y_k - 1) - 1]^2 - r_x^2 r_y^2 - [r_y^2(x_k + \frac{1}{2})^2 + r_x^2(y_k - 1)^2 - r_x^2 r_y^2] = r_y^2(x_{k+1} + \frac{1}{2})^2 + r_x^2(y_k - 1)^2 - 2r_x^2(y_k - 1) + r_x^2 - r_x^2 r_y^2 - r_y^2(x_k + \frac{1}{2})^2 - r_x^2(y_k - 1)^2 + r_x^2 r_y^2$$

$$P2_{k+1} = P2_k + r_y^2(x_{k+1} + \frac{1}{2})^2 - 2r_x^2(y_k - 1) + r_x^2 - r_y^2(x_k + \frac{1}{2})^2$$

where x_{k+1} is either x_k or $x_k + 1$ depending on the sign of $P2_k$.

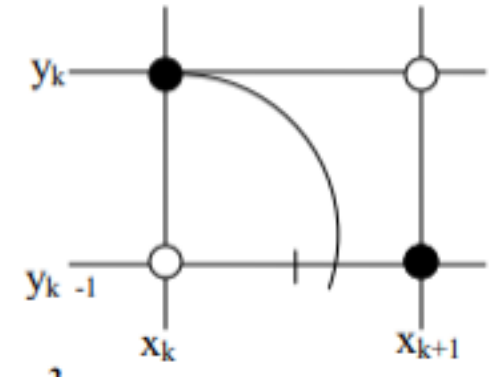
Ellipse

Case 1:

If $P2_k \leq 0$ then the mid point is inside the boundary of the ellipse, so we select the pixel at ' $x_k + 1$ '

$$P2_{k+1} = P2_k + r_y^2(x_k + 1 + \frac{1}{2})^2 - 2r_x^2(y_k - 1) + r_x^2 - r_y^2(x_k + \frac{1}{2})^2 = P2_k + r_y^2[(x_k + \frac{1}{2})^2 + 2(x_k + \frac{1}{2}) + 1] - 2r_x^2(y_k - 1) + r_x^2 - r_y^2(x_k + \frac{1}{2})^2 = P2_k + r_y^2(x_k + \frac{1}{2})^2 + 2r_y^2(x_k + \frac{1}{2}) + r_y^2 - 2r_x^2(y_k - 1) + r_x^2 - r_y^2(x_k + \frac{1}{2})^2 = P2_k + 2r_y^2 x_k + r_y^2 + r_y^2 - 2r_x^2(y_k - 1) + r_x^2$$

$$= P2_k + 2r_y^2(x_k + 1) - 2r_x^2(y_k - 1) + r_x^2$$



$$= P2_k + 2r_y^2(x_{k+1}) - 2r_x^2 y_{k-1} + r_x^2$$

Ellipse

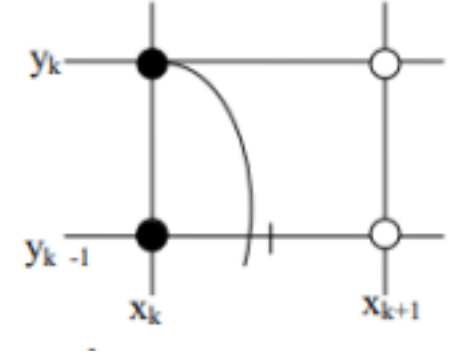
Case 2:

if $P2_k > 0$ then the mid point is outside the boundary of the ellipse, so we select the pixel at ' x_k '

$$\begin{aligned} P2_{k+1} &= P2_k + r_y^2(x_k + \frac{1}{2})^2 - 2r_x^2(y_k - 1) + r_x^2 - r_y^2(x_k + \frac{1}{2})^2 \\ &= P2_k - 2r_x^2(y_{k+1}) + r_x^2 \end{aligned}$$

For region 2, the initial position (x_0, y_0) is taken as the last position selected in region 1 and thus the initial decision parameter in region 2 is: $P2_0 =$

$$\begin{aligned} F_{\text{ellipse}}(x_0 + \frac{1}{2}, y_0 - 1) &= \\ r_y^2(x_0 + \frac{1}{2})^2 + r_x^2(y_0 - 1)^2 - r_x^2 r_y^2 \end{aligned}$$



Thank you

COMPUTER GRAPHICS (CAC305) LECTURE 10

Er Sanim Kumar Khatri

CONTENT

1.6 Area Filling

FILLED-AREA PRIMITIVES

Objects can be filled with solid-color or patterned polygon area in some graphical packages. Standard output primitive in graphics packages is solid color ,patterned polygon area
Polygons are easier to process due to linear boundaries.

Two basic approaches to area filling on a raster system

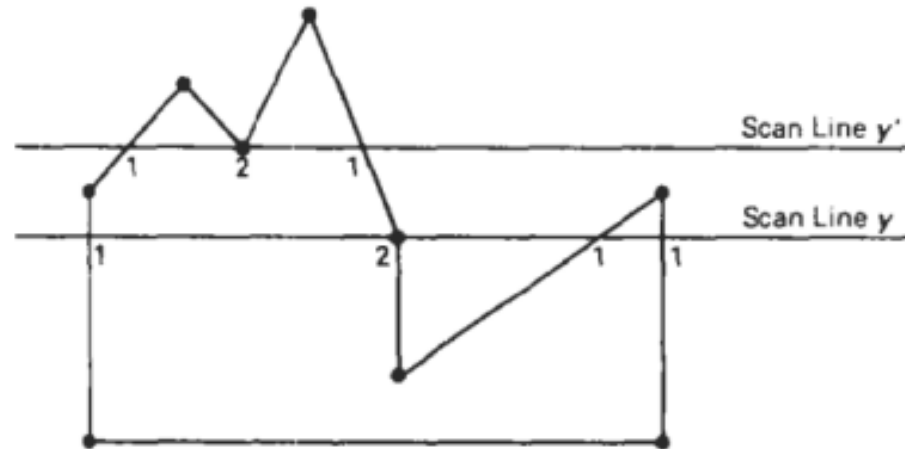
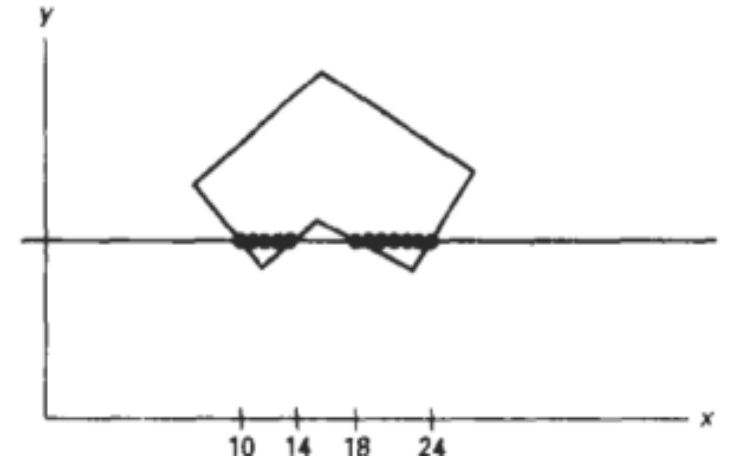
- i) Determine the overlap intervals for scan lines that cross the area. Typically useful for filling polygons, circles, ellipses
- ii) Start from a given interior position and paint outwards from this point until we encounter the specified boundary conditions useful for filling more complex boundaries, interactive painting system.

Two things to consider

- i. which pixels to fill
- ii. with what value to fill

Scan line Polygon Fill Algorithm

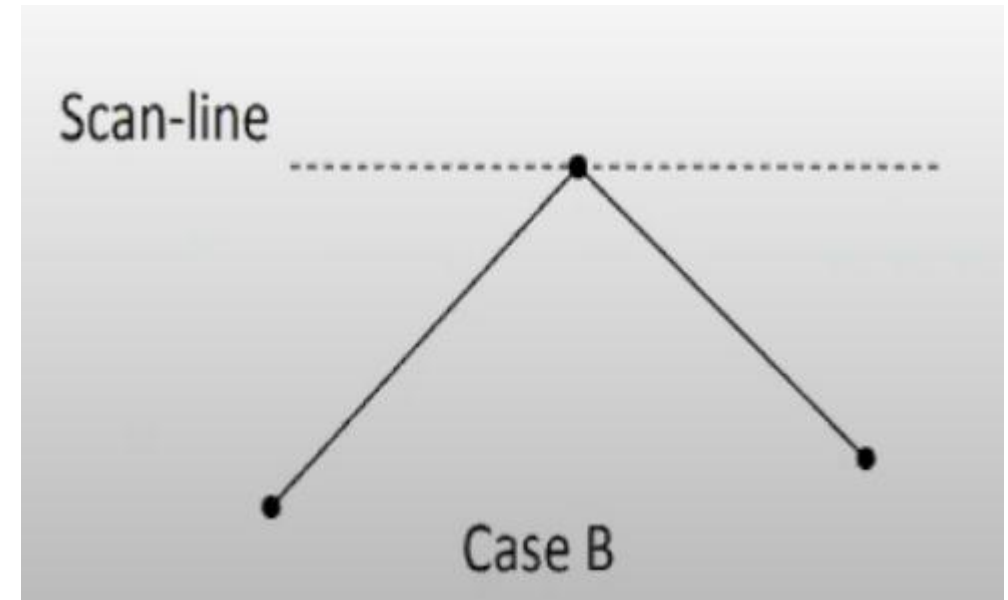
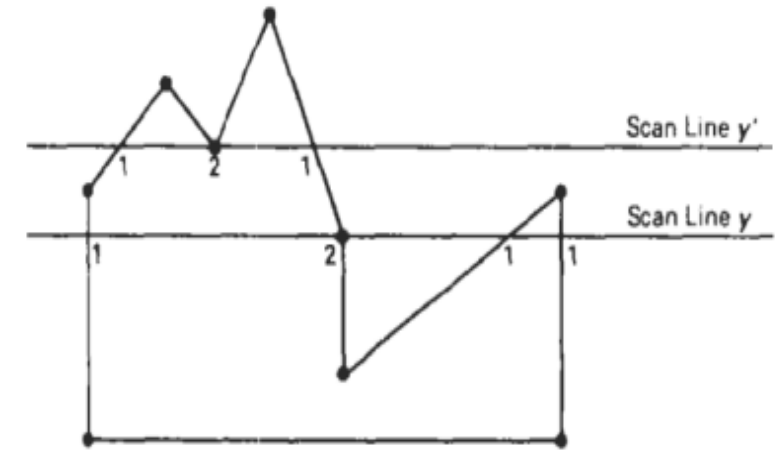
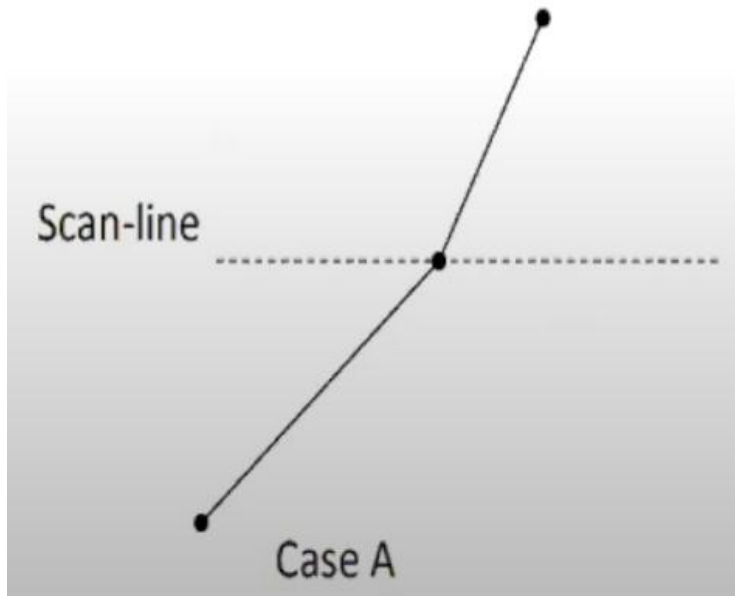
For each scan line crossing a polygon, the area-fill algorithm locates the intersection points of the scan line with the polygon edges. These intersection points are then sorted from left to right, and the corresponding frame-buffer positions between each intersection pair are set to the specified fill color. Four pixel intersection positions with the polygon boundaries define two stretches of interior pixels from $x = 10$ to $x = 14$ and from $x = 18$ to $x = 24$. The number of intersection points must be even.



Scan line Polygon Fill Algorithm

When the scan line passes through the vertex, we count it as 2 or 1 depending on the following rules.

- 1) If the edges of the polygon lie on the different sides, count it as only once.
- 2) If the edges of the polygon lie on the same side, count it as two.



Boundary Fill Algorithm

Starts at a point inside a region and paint the interior outward toward the boundary.

If the boundary is specified in a single color, the fill algorithm proceeds outward pixel by pixel until the boundary color is encountered.

This approach requires boundary to be specified, but interior pixels can be of multiple colors.

Four connected approach considers neighboring pixels as the ones that are to the right, left, bottom and top of the seed pixel.

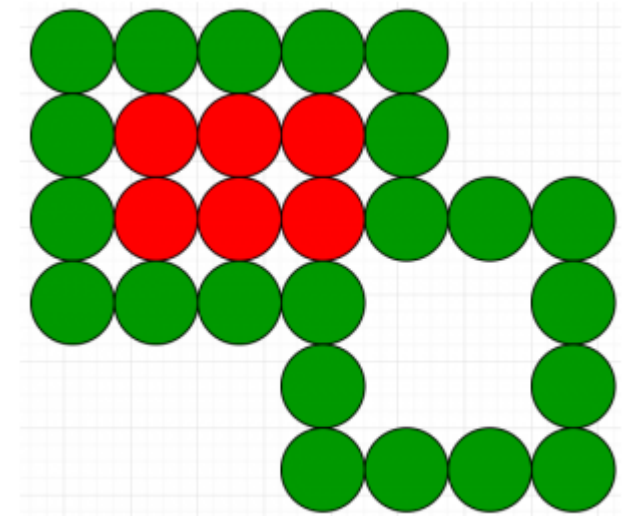
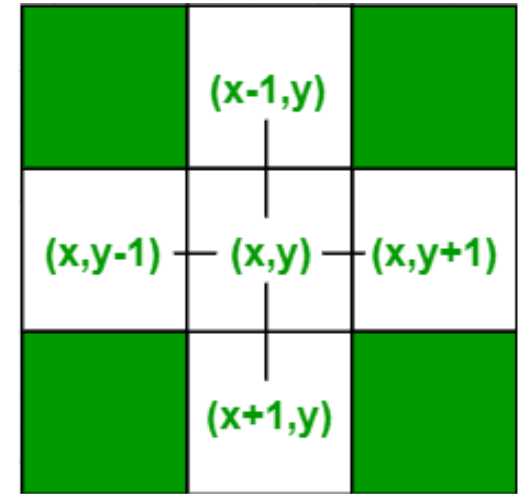
Eight connected approach also considers diagonal pixels as neighboring pixels so it includes pixels to the right, left, bottom , top then four more diagonal ones right-top, right-bottom, left-top , left-bottom

Boundary Fill Algorithm

Algorithm

Four connected approach

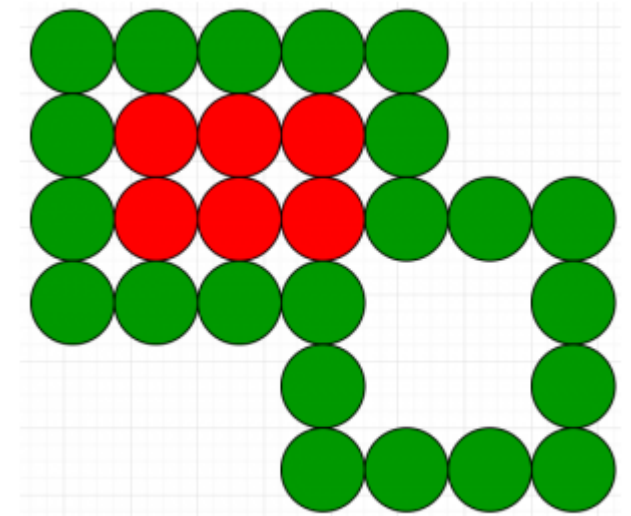
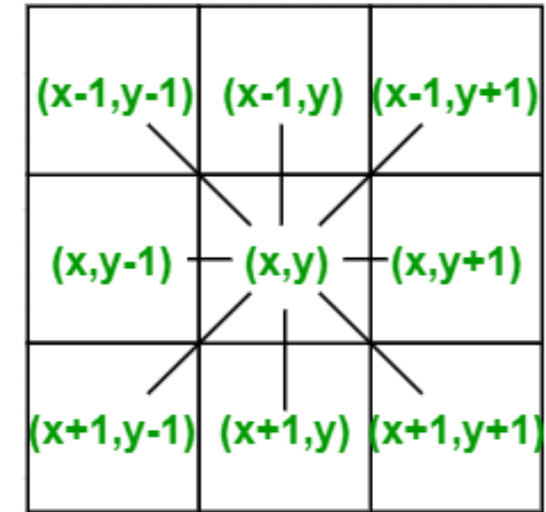
```
void boundaryFill (int x, int y, int fill, int boundary)
{
    int current = getpixel (x, y);
    if ((current != boundary) && (current != fill))
    {
        setpixel (x, y, fill);
        boundaryfill (x+1, y, fill, boundary);
        boundaryFill (x-1, y, fill, boundary);
        boundaryFill (x, y+1, fill, boundary);
        boundaryFill (x, y-1, fill, boundary);
    }
}
```



Boundary Fill Algorithm

Eight connected approach

```
void boundaryFill (int x, int y, int fill, int boundary)
{
    int current = getpixel (x, y);
    if ((current != boundary) && (current != fill))
    {
        setpixel (x, y, fill);
        boundaryfill (x+1, y, fill, boundary);
        boundaryFill (x-1, y, fill, boundary);
        boundaryFill (x, y+1, fill, boundary);
        boundaryFill (x, y-1, fill, boundary);
        boundaryFill (x-1, y-1, fill, boundary);
        boundaryFill (x-1, y+1, fill, boundary);
        boundaryFill (x+1, y-1, fill, boundary);
        boundaryFill (x+1, y+1, fill, boundary);
    }
}
```



Flood-Fill Algorithm

Used for filling an area that is not defined within a single color boundary.

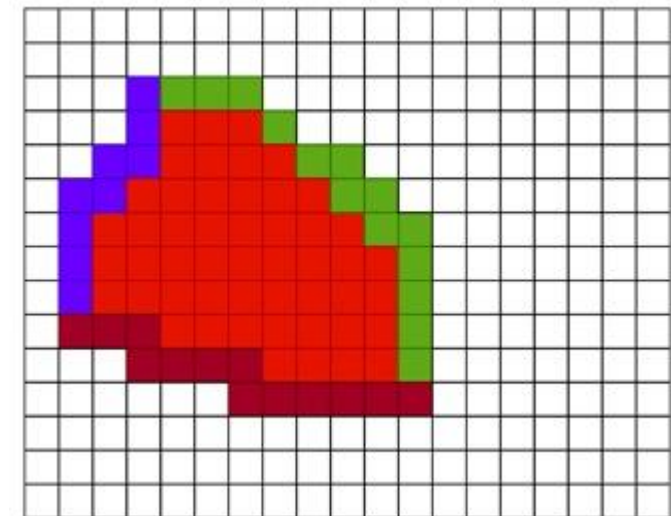
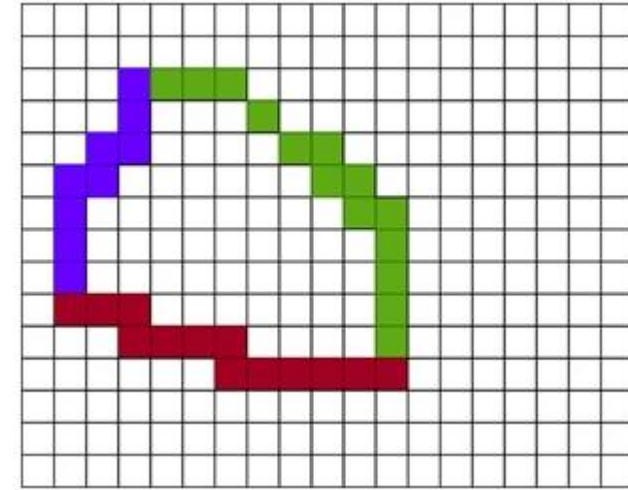
Such areas can be filled by replacing a specified interior color instead of searching for a boundary color value.

This approach does not require boundary to be specified, but interior pixels must be of the same color

This also has 2 methods, as 4 connected approach or 8 connected approach.

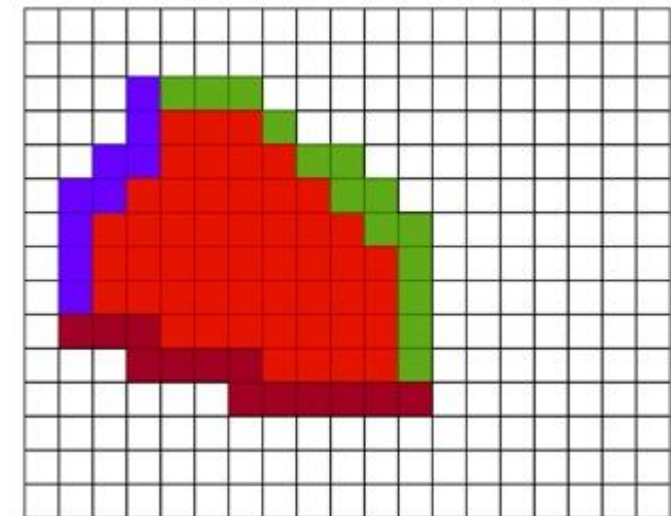
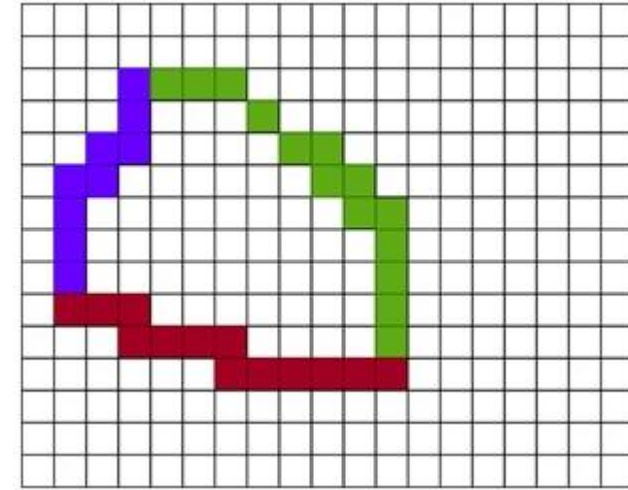
Flood Fill Algorithm

```
void floodFill (int x, int y, int fillcolor, int oldcolor)
{
    if (getpixel (x, y) == oldcolor)
    {
        setpixel (x, y, fillcolor);
        floodFill (x+1, y, fillColor, oldColor);
        floodfill (x-1, y, fillcolor, oldcolor);
        floodFill (x, y+1, fillcolor, oldcolor);
        floodFill (x, y-1, fillColor, oldcolor);
    }
}
```



Flood Fill Algorithm

```
void floodFill (int x, int y, int fillcolor, int oldcolor)
{
    if (getpixel (x, y) == oldcolor)
    {
        setpixel (x, y, fillcolor);
        floodFill (x+1, y, fillColor, oldColor);
        floodfill (x-1, y, fillcolor, oldcolor);
        floodFill (x, y+1, fillcolor, oldcolor);
        floodFill (x, y-1, fillColor, oldcolor);
    }
}
```



Thank you